

PRESTO: PERSPECTIVE-BASED ENTROPY-SPARSITY TRAINING OPTIMIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Pruning and quantization are the two standard ways of making a neural network smaller, but the entropy of the representation that they produce is rarely part of what training optimizes. We introduce PRESTO, a quantization-aware training method that trains against a surrogate, defined through a convex inner problem, of the size the weights will occupy after lossless coding. Uniform quantization with learned step sizes is combined with an inner problem, in perspective form, that trades the mass each weight places on the non-zero symbols against a convex penalty on their relaxed counts, the two channels into which that coded size decomposes. Because those counts couple every weight of a tensor, we relax the count constraints in a Lagrangian way, which separates the inner problem into one subproblem per symbol and one per weight, both solved exactly at a fixed multiplier at a cost linear in the number of symbols. The envelope derivatives of the relaxed value function then give gradients for the latent weights and for the step sizes, so that the network, the grid and the symbol distribution move together, and since the dual is advanced by a few warm-started steps rather than solved, that direction is an exact subgradient of a Lagrangian lower bound of the regularizer as the solver poses it. Measured against matched controls, PRESTO reduces the size of a quantized network while its accuracy stays above the full-precision checkpoint, reaching 4.89% of the checkpoint on ResNet-18 and 5.12% on ResNet-50 against controls at 9.84% and 9.97% trained on the same budget, which the recipe without the regularizer instead makes larger, and 4.24% on ResNet-18 once the schedule is extended to a hundred epochs. On the architectures whose control already falls below full precision we report the frontier against that control: DeiT-Small reaches 9.06% from a control at 11.99%, EfficientNet-B0 reaches 10.15% at 76.820 top-1 where the reference implementation of ISO/IEC 15938-17 reports 76.780 at 11.25%, and ViT-B/16 delivers 80.372% top-1 at 4.41% of the checkpoint where the same codec reports 75.22% at 4.85%.

1 INTRODUCTION

Model compression works through three complementary mechanisms, since pruning removes parameters, quantization represents the surviving values with a finite codebook, and lossless coding removes the statistical redundancy that is left in the resulting stream of symbols. The classical Deep Compression pipeline applies the three in sequence (Han et al., 2016), and its success raises the design question that we take up here: should a network be compressed after it has been trained, or should its compressibility shape the training objective itself?

Quantization-aware training follows the second principle by simulating low-precision arithmetic while the network is being optimized, so that PACT learns the clipping range of the activations (Choi et al., 2018) and LSQ optimizes the quantization scales directly (Esser et al., 2020), while post-training methods such as HAWQ (Dong et al., 2019), BRECQ (Li et al., 2021) and L² (Shi et al., 2023) avoid retraining altogether by reconstructing each layer locally. All of these methods control distortion or hardware precision, and none of them controls entropy, although a representation with few bits per weight need not have low entropy: two models that use the same codebook can produce symbol streams whose compressed sizes are very different. Entropy-aware compression addresses that difference explicitly. Entropy-constrained training writes the size of the model into the objective (Wiedemann et al., 2018), DeepCABAC combines rate-distortion quantization with context-adaptive

coding (Wiedemann et al., 2019), and the same coding engine has since become ISO/IEC 15938-17 and the open-source NNCodec (Becking et al., 2023), which is the natural reference for any claim about size after lossless coding, while CERWU (Conzelmann and Bamler, 2025) improves on that standard by 20 to 40% in bit rate on convolutional networks. Code length is therefore an established target rather than a by-product of the nominal bit width, but all of these methods act after training has finished, on a checkpoint that they did not shape.

A second family of methods shapes the representation while the network is being trained. Entropy penalized reparameterization (Oktay et al., 2020) learns the weights in a latent space that has its own probability model and entropy penalty, and on ResNet-18 it is, as far as we know, the smallest lossless entropy-coded model that has been published, at 4.22% of the checkpoint, while LilNetX (Girish et al., 2023) adds structured sparsity to the same formulation and reaches 2.16% at 1.2 points below its own baseline. Both of them rebuild a dense full-precision network at deployment time, so they save storage without producing a low-precision model, whereas PRESTO returns a genuine four-bit codebook, eight bits on the few tensors listed in Section 4, whose symbol stream is what the encoder actually sees. The closest method to our objective is HEMP (Tartaglione et al., 2021), which adds a differentiable entropy term to the loss. PRESTO differs from it in three ways: the inner problem is solved exactly at a fixed multiplier instead of being smoothed, so the gradient is an envelope derivative of the relaxed value function rather than of a surrogate; sparsity and the symbol distribution are controlled by a single relaxation instead of an entropy term paired with a separate pruning stage; and the same derivatives are applied to the learned step sizes, so the grid moves along with the weights. In concurrent work, CoDeQ (Wenshøj et al., 2025) observes that the dead zone of a scalar quantizer is exactly magnitude pruning and learns its width by backpropagation, which is the mechanism that our sparsity term also uses, although it reports bit operations rather than coded size. Learning the accuracy, the sparsity, the quantization levels and the symbol distribution at the same time remains difficult, because the symbol counts couple every weight that shares a codebook.

PRESTO handles that coupling with perspective optimization. Perspective reformulations have already produced effective regularizers for structured pruning (Cacciola et al., 2023), and we extend the idea from a binary keep-or-remove decision to a relaxed assignment over the quantization symbols, so that the inner problem represents each latent weight, controls the mass that the weight places on non-zero symbols and aggregates the global symbol counts. A Lagrangian relaxation of the count constraints then splits the tensor-level problem into per-symbol and per-weight subproblems that can be solved in parallel, while the envelope derivatives make the resulting value function usable inside ordinary gradient-based training once the dual has been advanced.

Our contributions are the following. We formulate a perspective, entropy-aware regularizer that couples sparsity and the symbol distribution inside learned-step quantization, and we derive a Lagrangian decomposition whose inner problem is solved exactly in closed form at $O(C)$ per weight, with envelope derivatives for the latent weights and for the learned step sizes and an explicit account of what those derivatives are exact for when the dual is truncated. Measuring against matched controls on the five architectures of Table 1, we show that the regularizer halves the serialized size of a quantized network while its accuracy stays above the full-precision checkpoint, although the same training without the regularizer makes the model larger. A term ablation on two architectures then shows that which of the two coefficients is sufficient on its own is a property of the architecture and not of the method, since the sparsity term alone recovers 97.5% of the compression on AlexNet but only 58.1% on ResNet-18, where the entropy term instead recovers 97.2%. We also report what the method does not do, including the one architecture on which published work is still ahead of us and our diagnosis of the reason.

Outline. Sections 2 and 3 state the method and its optimization, and Section 4 reports the experiments. Appendix A documents the training implementation and every run: its configuration, the quantized fraction of each checkpoint, the two term-ablation tables, the cost of the support mask, notes on the transfer architectures, the wall-clock cost and the learning curves. Appendix B reports the remaining experiments in full, from the measurement that quantization-aware training on its own makes models larger to the comparison between the joint objective and the same objective split into two stages, the transformer frontiers, the two mechanisms, the cost of per-channel grids and the negative results. Appendix C gives every derivation, Appendix D the procedure by which the coefficients are calibrated, Appendix E what is held identical across architectures and what is not, and Appendix F the glossary of notation.

2 PRESTO METHOD

2.1 SETTING, NOTATION AND THE QUANTIZER

Let $f(\cdot; \theta)$ be a network whose parameters divide into the weight tensors that we quantize, indexed by Λ , and the remaining floating-point parameters θ_{fp} , which are the biases and the normalization parameters. Tensor $\ell \in \Lambda$ has n_ℓ entries indexed by $i \in \mathcal{I}_\ell$ and latent weights $w_i^{(\ell)}$, and tensor ℓ uses a codebook of cardinality C_ℓ , of which $\mathcal{B}$ denotes the $B_\ell := C_\ell - 1$ non-zero symbols; we drop the subscript wherever one tensor is under discussion, which is everywhere below. We keep the zero symbol separate from that set, because the relaxation of Section 2.2 represents it by a mass $z_i^{(\ell)}$ instead of by an element of $\mathcal{B}$. The task loss $\mathcal{L}_{\text{task}}$ is the usual cross-entropy risk, and Appendix F lists every symbol that we use.

We build on LSQ (Esser et al., 2020), in which each tensor carries a single positive step size a_ℓ that is learned together with the weights. A latent weight is mapped onto the integer grid by $r_i^{(\ell)} := w_i^{(\ell)}/a_\ell$, then clipped to $[Q_n, Q_p]$, rounded and scaled back, so that the value that is deployed is $\hat{w}_i^{(\ell)} = a_\ell \text{clip}(\lfloor r_i^{(\ell)} \rfloor, Q_n, Q_p)$ and the clipped latent weight is $\tilde{w}_i^{(\ell)} := a_\ell \text{clip}(r_i^{(\ell)}, Q_n, Q_p)$. A per-channel variant, in which a_ℓ becomes one step size per output channel, is used only by the superseded controls of Table 4 and is priced in Appendix B.7; every run of Tables 1 to 6 is per-tensor. The rounding is to nearest with ties to even and is handled in the backward pass by a straight-through estimator. With that convention the deployed value is the zero symbol exactly when

$$\left| w_i^{(\ell)} \right| \leq \frac{1}{2} a_\ell, \quad (1)$$

so that every zero reported in this paper is produced by that rule and by no external pruning step. Appendix C.1 gives the grid, the codebook values $v_b^{(\ell)} = a_\ell q_b$ and the initialization of a_ℓ in full.

2.2 PERSPECTIVE AND ENTROPY-BASED REGULARIZER

Define the set of nonzero symbols as

$$\mathcal{B} := \mathcal{Q} \setminus \{0\}, \quad B := |\mathcal{B}| = C - 1.$$

The letter b indexes an element of $\mathcal{B}$, and q_b is the corresponding integer code. In tensor ℓ , the real-valued level associated with symbol b is

$$v_b^{(\ell)}(a_\ell) := a_\ell q_b. \quad (2)$$

For every pair $(i, b) \in \mathcal{I}_\ell \times \mathcal{B}$ we introduce a non-negative assignment mass $x_{i,b}^{(\ell)}$, and write $y_i^{(\ell)} \in [0, 1]$ for the total mass with which weight i is represented by non-zero symbols, $z_i^{(\ell)} := 1 - y_i^{(\ell)}$ for the mass on the zero symbol, and $c_b^{(\ell)} \geq 0$ for the relaxed count of symbol b . Below, a_ℓ and $\bar{w}^{(\ell)}(a_\ell)$ are arguments while $x^{(\ell)}, y^{(\ell)}, c^{(\ell)}$ are the variables of the inner minimization, and $T_1, T_2, T_3 \geq 0$ are the three coefficients. We define¹

$$\Phi_\ell(\bar{w}^{(\ell)}, a_\ell) := \min_{x^{(\ell)}, y^{(\ell)}, c^{(\ell)}} T_1 \sum_{i \in \mathcal{I}_\ell} \frac{(\bar{w}_i^{(\ell)})^2}{y_i^{(\ell)}} + T_2 \sum_{i \in \mathcal{I}_\ell} y_i^{(\ell)} + T_3 \sum_{b \in \mathcal{B}} c_b^{(\ell)} \log_2 c_b^{(\ell)} \quad (3)$$

$$\text{s.t.} \quad \sum_{b \in \mathcal{B}} v_b^{(\ell)}(a_\ell) x_{i,b}^{(\ell)} = \bar{w}_i^{(\ell)}, \quad i \in \mathcal{I}_\ell, \quad (4)$$

$$\sum_{b \in \mathcal{B}} x_{i,b}^{(\ell)} = y_i^{(\ell)}, \quad i \in \mathcal{I}_\ell, \quad (5)$$

$$c_b^{(\ell)} = \sum_{i \in \mathcal{I}_\ell} x_{i,b}^{(\ell)}, \quad b \in \mathcal{B}, \quad (6)$$

$$x_{i,b}^{(\ell)} \geq 0, \quad 0 \leq y_i^{(\ell)} \leq 1, \quad i \in \mathcal{I}_\ell, b \in \mathcal{B}. \quad (7)$$

¹With the continuous extension $0 \log_2 0 := 0$.

The T_1 term uses the closed extension of u^2/y : it equals u^2/y for $y > 0$, is 0 at $(u, y) = (0, 0)$ and $+\infty$ when $y = 0$ with $u \neq 0$, which makes explicit that a weight can carry zero mass only when the weight itself is zero.

The three coefficients play different roles. T_1 weights the perspective ridge, T_2 penalizes the mass assigned to non-zero symbols, and T_3 weights the term we call the entropy term, by which we mean the convex penalty $\sum_b c_b^{(\ell)} \log_2 c_b^{(\ell)}$ on the relaxed counts and nothing more. It deserves a precise statement, because that sum is not the entropy of the non-zero symbols and minimizing it does not minimize that entropy. Writing $m = \sum_b c_b^{(\ell)}$, the ideal zero-order code length of a tensor is $n_\ell H(m/n_\ell) + mH(c^{(\ell)}/m)$, the mask plus the value stream, which is n_ℓ times the entropy of the distribution over all C_ℓ symbols with the zero symbol included, and it is a proxy for what the coder produces rather than that size itself (Appendix A.4). Since $c \mapsto c \log_2 c$ is minimized at $c = 1/e$, our term prices every non-zero symbol and pushes mass into the excluded zero symbol, which is the channel through which it lowers that entropy, while at a fixed m it acts weakly the other way because the same sum is minimized at uniform counts. Appendix C.11 gives the arithmetic. The values of the coefficients are not part of the definition of the method and are given only in the experimental protocol. The zero symbol is excluded from $\mathcal{B}$ and is represented only by $z_i^{(\ell)}$, which prevents the same zero value from having two different representations.

Inside the training procedure $z_i^{(\ell)*}$ is a relaxed variable of the inner problem rather than a hard pruning decision, so it contributes to the PRESTO subgradient but never overwrites a latent weight with zero: PRESTO reshapes the latent distribution towards the region that the quantizer maps to zero, and LSQ is what produces the zeros seen in the forward pass and in the deployed model.

The outer PRESTO objective is $\min_{w, a, \theta_{\text{fp}}} \mathcal{J}$ with

$$\mathcal{J}(w, a, \theta_{\text{fp}}) := \mathcal{L}_{\text{task}}(\hat{w}(a), \theta_{\text{fp}}) + \underbrace{\sum_{\ell \in \Lambda} \Phi_\ell(\bar{w}^{(\ell)}(a_\ell), a_\ell)}_{\mathcal{R}_{\text{PRESTO}}(w, a)}. \quad (8)$$

The task loss contributes its ordinary gradient, and the regularizer contributes a subgradient $\nabla_{\text{PRESTO}} \in \partial_{(w, a)} \mathcal{R}_{\text{PRESTO}} \times \{0\}$ in which the zero appears because $\mathcal{R}_{\text{PRESTO}}$ does not depend on the floating-point parameters, and the sum of the two lies in $\partial \mathcal{J}$. That is the direction we would follow if the inner problem were solved to optimality; Section 3 derives its components and states what a truncated dual delivers instead.

3 OPTIMIZATION

3.1 LAGRANGIAN RELAXATION AND DECOMPOSITION

The inner problem of Eq. (3) contains one set of variables for every weight and a second set that is shared across the tensor. The representation and mass constraints of Eqs. (4) and (5) involve a single weight index i , whereas each count constraint of Eq. (6) couples all the weights through the sum $c_b^{(\ell)} = \sum_{i \in \mathcal{I}_\ell} x_{i,b}^{(\ell)}$, so that solving the inner problem directly would mean handling every assignment mass of a tensor at once. Lagrangian relaxation removes exactly that global coupling.

Fix a tensor $\ell \in \Lambda$, a clipped latent-weight vector $\bar{w}^{(\ell)}$ and a step size a_ℓ . Attaching an unrestricted multiplier $\xi_b^{(\ell)} \in \mathbb{R}$ to each count constraint and dualizing only those constraints gives a partial Lagrangian whose minimum over the remaining feasible variables is the dual function $\varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}(\xi^{(\ell)})$, concave as a pointwise infimum of affine functions of the multipliers. Appendix C.2 carries the algebra; the result is that it splits:

$$\begin{aligned} \varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}(\xi^{(\ell)}) = & \sum_{b \in \mathcal{B}} \min_{c_b^{(\ell)} \geq 0} \left\{ T_3 c_b^{(\ell)} \log_2 c_b^{(\ell)} - \xi_b^{(\ell)} c_b^{(\ell)} \right\} \\ & + \sum_{i \in \mathcal{I}_\ell} \min_{(x_i^{(\ell)}, y_i^{(\ell)}) \in \mathcal{F}_i^{(\ell)}} \left\{ T_1 \frac{(\bar{w}_i^{(\ell)})^2}{y_i^{(\ell)}} + T_2 y_i^{(\ell)} + \sum_{b \in \mathcal{B}} \xi_b^{(\ell)} x_{i,b}^{(\ell)} \right\}, \end{aligned} \quad (9)$$

where $x_i^{(\ell)} = (x_{i,b}^{(\ell)})_{b \in \mathcal{B}}$, and $\mathcal{F}_i^{(\ell)}$ is the local feasible set

$$\mathcal{F}_i^{(\ell)} := \left\{ (x_i^{(\ell)}, y_i^{(\ell)}) : \begin{array}{l} x_{i,b}^{(\ell)} \geq 0 \quad \text{for every } b \in \mathcal{B}, \\ 0 \leq y_i^{(\ell)} \leq 1, \\ \sum_{b \in \mathcal{B}} v_b^{(\ell)}(a_\ell) x_{i,b}^{(\ell)} = \bar{w}_i^{(\ell)}, \\ \sum_{b \in \mathcal{B}} x_{i,b}^{(\ell)} = y_i^{(\ell)} \end{array} \right\}. \quad (10)$$

The first line consists of B independent one-dimensional count subproblems, one for each non-zero symbol, and the second consists of n_ℓ independent per-weight subproblems that share the multiplier but no optimization variables, so that once $\xi^{(\ell)}$ is fixed the tensor-level problem can be solved in parallel across symbols and across weights, which is the reason for relaxing it in the first place. The dual problem is

$$\max_{\xi^{(\ell)} \in \mathbb{R}^B} \varphi_{\bar{w}^{(\ell)}, a_\ell}(\xi^{(\ell)}), \quad (11)$$

and it is exact: the objective is convex and closed, the constraints are affine and the feasible set is compact, so the value of the perturbed problem is lower semicontinuous at the origin and no duality gap remains. Proposition 1, in Appendix C.10, proves this and separates the two roles the hypotheses play, since clipping the latent weights is what makes the problem feasible with finite value while a single weight strictly inside the representable interval is what makes the dual optimum attained. The relaxation therefore changes the way the inner problem is solved without changing its value. Finally, for a fixed $\xi^{(\ell)}$ the difference between the counts that the per-weight solutions reconstruct and the counts that the count subproblems return is a supergradient of $\varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}$, and that difference drives the dual iteration described below.

3.2 PER-WEIGHT SOLUTION, GRADIENTS AND DUAL UPDATE

For a fixed multiplier the per-weight subproblem is a linear knapsack over the B non-zero symbols together with the zero symbol, and it is solved exactly in closed form. Every candidate symbol contributes a pair (v_b, ξ_b) , the optimum is reached at a vertex of the lower convex hull of those pairs, so the assignment mass concentrates on at most two adjacent symbols, and the optimal non-zero mass y_i^* then follows from a one-dimensional minimization of a perspective term against a linear one, at a cost of $O(C)$ per weight and with no iteration at all. Appendix C.3 states the breakpoints, proves that the vertices of the hull are the only candidates and gives the closed form of y_i^* , and Algorithm 1, in Appendix C.15, is the solver that follows from it.

At a fixed multiplier the two families of subproblems are solved exactly, so Danskin’s theorem applies to the inner minimization and the derivative that enters training is an *envelope* derivative evaluated at the inner optimum rather than the derivative of a smoothed surrogate. For the latent weights it is

$$\beta_i^* = \mu_i + \frac{2T_1 \bar{w}_i}{y_i^*}, \quad (12)$$

where μ_i collects the multiplier of the symbols to which the weight is assigned, and the step sizes receive the matching derivative through the dependence of v_b on a_ℓ , so that the network, the quantization grid and the symbol distribution all move within one backward pass (Appendices C.4–C.13).

Two qualifications belong with that statement. Because we advance the dual by N_{dual} warm-started steps per call rather than solving it, the expression is an exact subgradient of the *relaxed* value function at the multiplier we hold and an inexact proxy for Φ_ℓ , the count residual being our diagnostic for the gap rather than a bound on it; and the solver boxes the relaxed counts and projects the multiplier onto the matching interval, so the problem it poses is not literally Eq. (3) and the ascent runs on the projected dual of that boxed problem. Appendix C.12 states both differences precisely.

The multiplier itself is updated by normalized supergradient ascent with warm starts, for N_{dual} iterations every `entropy_every` optimizer steps, using the count residual divided by n_ℓ as the supergradient. One property of that update deserves attention, because the interval the multiplier has to cross is proportional to T_3 while a step of fixed absolute size is not, so raising the entropy coefficient moves the target away just as fast as it raises it and the term becomes weaker rather than stronger. Measuring the step as a fraction of the dual range removes that dependence and every run

in this paper uses the relative step; Appendix C.9 derives the rate and Appendix B.8 reports the runs on which we found the problem.

4 EXPERIMENTS

4.1 EVALUATION PROTOCOL

The question we ask throughout is whether PRESTO makes quantized weights smaller after loss-less coding at the same accuracy and the same training budget. Every run starts from a public full-precision checkpoint, quantizes all the convolutional and linear weight tensors and keeps the activations in FP32, and we never apply a pruning rule, so every zero is produced by Eq. (1) and the sparsity is emergent. The training machinery around that objective, the optimizers and schedules, the distillation and the batch-norm recalibration, is implementation rather than method and is stated in Appendix A.1. The codebook has $C_\ell = 16$ symbols everywhere except on the two input convolutions of AlexNet and the input and depthwise convolutions of EfficientNet-B0, 0.07% and 3.7% of their weights, which use eight bits and are counted at their true cost in every ratio. Each number is the final epoch of a single run unless we say otherwise.

Size is measured rather than nominal, and over the whole checkpoint. The *packet* is the complete serialized artifact, containing the support mask, the stream of non-zero symbols and the metadata, the learned step sizes included; a nominal bit width would hide both the gain from entropy coding and the cost of the metadata. Published methods report megabytes for the whole checkpoint, so we do the same: if f is the fraction of parameters we quantize, a packet of r against the quantized weights is $rf + (1 - f)$ of the checkpoint. In our networks f ranges from 99.01% to 99.98%; Table 3 lists it per network with the residue and with the packed attention parameter of ViT-B/16, which has to be quantized as well for the ratios to stay comparable.

Accuracies are compared raw rather than as drops. Every method measures its own baseline and the same checkpoint evaluated by two harnesses does not give the same number: our 69.734 on ResNet-18 is 0.266 below the 70.0 of Oktay et al. (2020), and DeiT-Small and EfficientNet-B0 go the same way. Comparing drops would credit us with those differences, so we compare top-1 directly wherever the checkpoint is shared and fall back on drops only for ResNet-50 against NNCodec. Each frontier is a one-parameter family: on the ResNets we vary T_3 at fixed T_2 , on the transformers we scale the pair along the ray $T_2/T_3 = 10/3$.

4.2 CONTROLLED COMPARISON ON RESNET-18

Table 1 contains the central measurement, and Figure 2 in Appendix B places every run of this paper and every published point within its range on a single panel. Within each block the rows share the checkpoint, the schedule, the codebook and the byte accounting, and differ only in the triple (T_1, T_2, T_3) and, where stated, in the budget.

PRESTO halves the packet of the LSQ control, from 9.84% of the checkpoint to 4.89%, and still finishes 0.286 points above full precision. That row is the mean of three repeats of an identical configuration, with standard deviations of 0.083 on the accuracy and 0.015 on the packet, so the margin is 3.4 standard deviations; ResNet-50 reproduces the trade at a third of the budget, halving its control to 5.12% at 0.196 above its checkpoint. One reading of these rows has to be resisted, since the LSQ control itself finishes 0.756 above the checkpoint and clearing full precision is therefore not by itself a demanding bar on this network; the claim we make is the controlled one, that the regularizer halves the size of a quantized ResNet while its accuracy stays above the checkpoint that the control also clears.

Budget, rather than dose, improves both axes at once: at $T_3 = 10^{-7}$ the run lands 0.128 below the checkpoint at sixty epochs and 0.126 *above* it at one hundred, at the same size and still gaining when the schedule ended. Raising T_3 instead trades one axis against the other, and the frontier bends: the marginal cost per point of size is 0.625 accuracy points between the sixty-epoch rows of Tables 1 and 6 and 1.255 between the two hundred-epoch ones. Appendix B.2 gives both readings in full.

Table 1: ImageNet. *Size* is the serialized packet over the whole checkpoint, *Top-1* is raw accuracy. Our rows within a block differ only in (T_1, T_2, T_3) and, where stated, the budget; published rows are converted from the megabytes they report, with their own measurement of the same checkpoint in brackets where it differs from ours. More points in Table 6. *nominal size, no entropy coding. [‡]reconstructs a dense full-precision network at deployment. [§]a different checkpoint, so only the drop is comparable.

Network	Variant	Ep.	Top-1	Size	Spars.
ResNet-18 FP32 69.734	LSQ only	60	70.490	9.84%	18.2%
	PRESTO ($10^{-5}, 10^{-7}, 6 \cdot 10^{-8}$), 3 repeats	60	70.020 $\pm$.083	4.89% $\pm$.015	61.8%
	PRESTO ($10^{-5}, 10^{-7}, 10^{-7}$)	100	69.860	4.24%	69.7%
	HEMP + LOBSTER (Tartaglione et al., 2021)	—	69.70 [69.76]	5.34%	—
	EPR-DFT [‡] (Oktay et al., 2020)	—	70.0 [70.0]	4.22%	—
ResNet-50 FP32 76.122	LSQ only	20	76.554	9.97%	22.1%
	PRESTO ($10^{-5}, 10^{-7}, 3 \cdot 10^{-8}$)	20	76.318	5.12%	60.0%
	HEMP (Tartaglione et al., 2021)	—	74.52 [76.13]	8.88%	—
	NNCodec [§] (Becking et al., 2023)	—	—1.01 [80.34]	7.43%	—
DeiT-Small FP32 79.742	LSQ + distillation	25	79.706	11.99%	13.7%
	PRESTO , reference dose	25	79.378	9.06%	39.7%
	PRESTO, twice the dose	25	78.948	7.74%	48.8%
	Q-ViT 3-bit (Li et al., 2022)	300	79.0 [79.9]	9.86%*	—
	FTerViT W2A8 (Rucinski et al., 2026)	—	77.47 [79.86]	6.59%*	—
ViT-B/16 FP32 81.056	LSQ + distillation	20	80.816	10.48%	21.8%
	PRESTO , reference dose	20	80.372	4.41%	72.9%
	NNCodec (Becking et al., 2023)	—	79.95 [81.07]	9.49%	—
	NNCodec (Becking et al., 2023)	—	75.22 [81.07]	4.85%	—
EfficientNet-B0 FP32 77.566	LSQ + distillation	30	77.034	12.05%	16.5%
	PRESTO ($10^{-5}, 2 \cdot 10^{-7}, 6 \cdot 10^{-8}$)	30	76.820	10.15%	28.0%
	NNCodec (Becking et al., 2023)	—	76.78 [77.67]	11.25%	—

4.3 TERM ABLATION: WHAT EACH TERM CONTRIBUTES

Five sixty-epoch runs on ResNet-18 and four on AlexNet, each set identical in every flag except the coefficient triple, are reported in Table 5, and we read them as the answer to one question: how much of the compression survives if only one term is kept? The ridge alone does nothing, giving 9.87% of the checkpoint against the control’s 9.84%, a difference the size of the spread between nominally identical runs.

Which of the two remaining terms suffices on its own is a property of the architecture rather than of the method. Writing the size reduction of the full objective as 100%, the shares a single term recovers are the following.

Network	size (% of checkpoint)				share recovered	
	control	T_2 only	T_3 only	full	T_2 only	T_3 only
ResNet-18	9.84%	6.96%	5.03%	4.89%	58.1%	97.2%
AlexNet	10.58%	3.04%	3.48%	2.85%	97.5%	91.8%

On AlexNet the sparsity term alone recovers 97.5% of the reduction and the entropy term adds the rest, whereas on ResNet-18 the same arm recovers 58.1% and the entropy term 97.2%; the shares are the same on the packet and on the whole checkpoint.

Both terms produce zeros, so the two arms are not a pruning channel and a coding channel. The entropy term alone reaches 73.72% sparsity on AlexNet and 60.30% on ResNet-18, past the 45.56% of the sparsity term there, which is what Section 2.2 predicts: the penalty prices every non-zero symbol, so its minimizer moves mass into the zero symbol. What separates the two is the second channel. Over a ResNet-18 run the sparsity term leaves the entropy of the non-zero symbols where it found it, at 2.502 to 2.526 bits, while the entropy term takes it from 2.784 to 1.926 and the control lets it rise to 3.215; we report that as a measurement and not as a property of the objective, since at a fixed non-zero mass the term favours uniform counts.

The two terms do not add. Against the better single arm the full objective is 2.8% smaller on ResNet-18 and 6.2% smaller on AlexNet, at accuracy differences of 0.070 and 0.126 points, inside the noise on ResNet-18 and unresolved on AlexNet, so the combination refines the size at no measurable cost in accuracy rather than multiplying it. In each block the full objective lands on the symbol-entropy floor of whichever term went lower, 1.902 bits against 1.926 on ResNet-18 and 1.686 against 1.688

on AlexNet, so the second term has little left to remove. We do not rank the two terms by size per accuracy point: those rates are secants to a concave frontier between arms that do not end at the same size, so the arm dosed further shows the flatter secant.

One asymmetry belongs with those numbers. With $T_3 = 0$ the regularizer is a closed-form gradient added at every optimizer step, whereas with $T_3 > 0$ it reaches the weights only when the dual runs, once every four steps and carrying all three terms, so the doses the shares compare are coefficient and frequency together. The bias runs against the entropy arms, applied four times less often and yet reaching further on ResNet-18.

The cost lies entirely in the entropy term. When $T_3 = 0$ the dual solver is never entered and the regularizer reduces to a closed-form per-step gradient, so the sparsity arm trains at 108 seconds per epoch on both networks, indistinguishable from the LSQ control, while the entropy arm costs $4.0\times$ on ResNet-18, where the full objective costs $4.1\times$, and $6.3\times$ on AlexNet. With the shares above this is a practical rule: on a redundant architecture the free term accounts for 97.5% of the compression and the last 2.5% costs six times the training, whereas without that redundancy the same factor of four separates a packet of 6.96% from one of 4.89%.

4.4 ALEXNET, AN ARCHITECTURE WHERE PRUNING-FIRST METHODS WIN

AlexNet is the one network here on which published work is still ahead of us, and the ablation explains why. 95.96% of its 61.1M weights sit in three fully connected tensors, one holding 62% alone, and redundancy on that scale is what a pruning pipeline exploits, which is why our sparsity term by itself recovers 97.5% of the compression here. Both methods ahead of us are prune-first. Deep Compression (Han et al., 2016), which combines magnitude pruning and iterative retraining with k -means weight sharing and Huffman coding, reports 2.86% of the checkpoint at parity with its baseline, while OPQ (Hu et al., 2021) reports +0.46 with a ratio counting only the surviving values that becomes 1.94% once the support mask is charged for. Our best point is 2.85% at 0.880 below full precision, so at Deep Compression’s size we are that much less accurate, and OPQ is smaller than both of us.

The gap is the mask rather than the coder. At 79.66% sparsity our packet divides into 1.973% of mask and 0.854% of values, whereas OPQ at 92.30% divides into 1.223% and 0.719%, so on the value stream we are the more compact, at a measured 1.34 bits per surviving weight against the 2.99 implied by their published bit width. The two are not the same kind of quantity, ours being what the coder produces and theirs what the codebook nominally costs (Appendix C.11). The whole difference is the support mask, whose i.i.d. cost model $H(p)/32$ falls steeply beyond $p = 0.8$, and we pay more of it only because we stop thirteen points of sparsity short; their figure is that model applied to their published sparsity and bit width rather than a measurement through our serializer, so it is an estimate. The codebook is not the explanation, since our sixteen-symbol grid is lossless here at -0.024 and 10.58% of the checkpoint.

What the deficit is and is not. It is neither the budget nor a healing phase too short to recover from the pruning, and what we believe it is instead is allocation: one coefficient pushes in the same way on eight tensors whose sizes span three orders of magnitude, while both methods ahead of us assign a rate per layer. Appendix B.6 gives the two measurements and the arithmetic of the allocation hypothesis, which we have not tested. We also have no converged measurement above 80% sparsity here, so whether our frontier reaches OPQ’s operating point is open, and we report AlexNet as a transfer architecture rather than a competitive row.

4.5 COMPARISON WITH PUBLISHED WORK

All the comparisons below use the whole-checkpoint accounting, and they use raw top-1 accuracy wherever the checkpoint is shared.

The ResNets. On ResNet-18 the strongest published point is EPR-DFT (Oktay et al., 2020), lossless at 4.22%, against our hundred-epoch 4.24% at 69.860. The sizes agree within the 0.015 standard deviation of our packet, and on accuracy the ordering depends on the baseline, since we are 0.126 above our checkpoint where they report parity with theirs and theirs is 0.266 higher, so we claim a

tie and note that we deliver a deployable four-bit network where EPR rebuilds a dense full-precision one through a learned decoder. Against HEMP with LOBSTER, at 69.70 and 5.34%, our headline row is more accurate and smaller. On ResNet-50 no competitor is on our frontier: our knee is 5.12% at 0.196 above the checkpoint against 7.43% at -1.01 for NNCodec and 8.88% at -1.61 for HEMP, and our frontier point reaches 3.72% at -0.678 where LilNetX needs 3.53% for -2.9 .

DeiT-Small. Q-ViT (Li et al., 2022) at three bits reports 79.0 at a nominal 9.86% against our measured 79.378 at 9.06%, so we are both more accurate and smaller, although they also quantize the activations. Their four-bit row, which Table 6 carries, is the stronger one, 80.9 against a 79.9 baseline, but at a nominal 12.93% that is larger than our own control and after three hundred epochs with their distillation scheme, so it is a point on neither of the two axes we are matching here; FTerViT (Rucinski et al., 2026) ternarizes to a bit-packed 6.59% at 2.39 below its baseline, where our 7.74% row is 0.794 below ours.

ViT-B/16. This is the one network for which the standardized codec publishes a full rate-accuracy grid. NNCodec needs 9.49% of the checkpoint to stay within 1.12 points, and at 4.85%, the size closest to ours, it delivers 75.22, whereas PRESTO delivers 80.372 at 4.41%: five accuracy points at a smaller size. The comparison is unusually clean, since both are weight-only, both measure bytes after lossless coding and both start from the same checkpoint.

EfficientNet-B0. Here the control lands almost a point below the checkpoint and there is no headroom at all, so a published method was ahead of us on this network for most of this work. Two diagnosed changes of recipe closed the gap: giving eight bits to the depthwise convolutions, which clipped 6.58% of their weights against 1.93% elsewhere, returned 0.362 accuracy points, and replacing the per-channel step sizes with per-tensor ones returned a further 1.03 points of packet, only 0.597 of which was metadata (Appendix B.7). PRESTO then delivers 76.820 at 10.15% against NNCodec’s 76.780 at 11.25%, smaller by a tenth of its size and, at essentially the same size, where their grid gives 76.29 at 10.08%, more accurate by 0.53; the accuracy margin on their headline point, 0.04, is inside the noise, the margin on size is not.

5 CONCLUSIONS AND FUTURE WORK

We presented PRESTO, a regularizer that trains a quantized network against a surrogate of its compressed size defined through a convex inner problem. It relaxes the assignment of each latent weight over the codebook, prices the resulting counts through a Lagrangian relaxation, and returns envelope derivatives for the weights and the learned step sizes. The inner problem remains convex, separable, and solvable in closed form at $O(C)$ per weight for a fixed multiplier. Across the architectures studied, PRESTO reduces the serialized weight packet relative to matched LSQ controls: from 9.84% to 4.89% on ResNet-18, from 9.97% to 5.12% on ResNet-50, from 11.99% to 9.06% on DeiT-Small, from 10.48% to 4.41% on ViT-B/16, and from 12.05% to 10.15% on EfficientNet-B0. These results show that entropy-aware training is effective beyond a single network family, while also making clear that the attainable trade-off depends on the architecture and on the training budget.

Several directions follow naturally from these findings. First, the number of codebook symbols C is currently selected before training. A further regularization term could instead penalize codebook complexity and learn an intelligent number of buckets jointly with the weights, allowing the model to adapt its representation capacity to each layer and architecture. Second, the method should be evaluated on substantially larger models, including modern LLMs, where the storage/compression trade-off and the cost of the inner optimization may behave differently. Third, the present experiments quantize weights while keeping activations in FP32; extending the objective and its accounting to activation quantization would enable end-to-end memory and inference-cost optimization. These extensions would clarify the scalability and deployment value of PRESTO without changing the central conclusion demonstrated here: learning the symbol distribution during quantized training can substantially reduce the coded model size at essentially preserved accuracy.

REFERENCES

- A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- D. Becking, P. Haase, K. Müller, H. Kirchhoffer, W. Samek, and D. Marpe. NNCodec: An open source software implementation of the neural network coding ISO/IEC standard. In *ICML Neural Compression Workshop*, 2023.
- Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- J. F. Bonnans and A. Shapiro. *Perturbation Analysis of Optimization Problems*. Springer, 2000.
- M. Cacciola, A. Frangioni, X. Li, and A. Lodi. Deep neural networks pruning via the structured perspective regularization. *SIAM Journal on Mathematics of Data Science*, 5(4):1051–1077, 2023.
- J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks. In *International Conference on Learning Representations*, 2018.
- A. Conzelmann and R. Bamler. Reducing storage of pretrained neural networks by rate-constrained quantization and entropy coding. *arXiv preprint arXiv:2505.18758*, 2025.
- J. M. Danskin. The theory of max-min, with applications. *SIAM Journal on Applied Mathematics*, 14(4):641–664, 1966.
- Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ: Hessian aware quantization of neural networks with mixed precision. In *IEEE/CVF International Conference on Computer Vision*, 2019.
- A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha. Learned step size quantization. In *International Conference on Learning Representations*, 2020.
- S. Girish, K. Gupta, S. Singh, and A. Shrivastava. LilNetX: Lightweight networks with extreme model compression and structured sparsification. In *International Conference on Learning Representations*, 2023.
- S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations*, 2016.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- P. Hu, X. Peng, H. Zhu, M. M. Sabry Aly, and J. Lin. OPQ: Compressing deep neural networks with one-shot pruning-quantization. In *AAAI Conference on Artificial Intelligence*, pp. 7780–7788, 2021.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2012.
- Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, and S. Gu. BRECQ: Pushing the limit of post-training quantization by block reconstruction. In *International Conference on Learning Representations*, 2021.
- Y. Li, S. Xu, B. Zhang, X. Cao, P. Gao, and G. Guo. Q-ViT: Accurate and fully quantized low-bit vision transformer. In *Advances in Neural Information Processing Systems*, 2022.
- C. Metz, O. Bichler, and A. Dupret. Efficient neural compression with inference-time decoding. In *IEEE International Symposium on Circuits and Systems*, 2024.
- D. Oktay, J. Ballé, S. Singh, and A. Shrivastava. Scalable model compression by entropy penalized reparameterization. In *International Conference on Learning Representations*, 2020.
- R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970.
- S. Rucinski, P. Bonazzi, E. Türetken, S. Narduzzi, M. Magno, and N. Maamari. FTerViT: Fully ternary vision transformer. *arXiv preprint arXiv:2605.21171*, 2026.
- Y. Shi, S. Bai, X. Wei, R. Gong, and J. Yang. Lossy and lossless (L^2) post-training model size compression. *arXiv preprint arXiv:2308.04269*, 2023.

- M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, 2019.
- E. Tartaglione, A. Bragagnolo, A. Odierna, A. Fiandrotti, and M. Grangetto. HEMP: High-order entropy minimization for neural network compression. *Neurocomputing*, 461:244–253, 2021.
- H. Touvron, M. Cord, M. Douze, M. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, 2021.
- J. Wenshøj, T. Chen, B. Pepin, and R. Selvan. CoDeQ: End-to-end joint model compression with dead-zone quantizer for high-sparsity and low-precision networks. *arXiv preprint arXiv:2512.12981*, 2025.
- S. Wiedemann, A. Marban, K.-R. Müller, and W. Samek. Entropy-constrained training of deep neural networks. *arXiv preprint arXiv:1812.07520*, 2018.
- S. Wiedemann, H. Kirchhoffer, S. Matlage, et al. DeepCABAC: Context-adaptive binary arithmetic coding for deep neural network compression. *arXiv preprint arXiv:1905.08318*, 2019.

A BEST-RUN CONFIGURATIONS AND REPRODUCIBILITY DETAILS

A.1 TRAINING IMPLEMENTATION

Schedules and teacher losses are choices of implementation rather than extra terms of the regularizer, so we keep them separate from the definition of PRESTO. We quantize the convolutional and linear weight tensors with a signed uniform LSQ codebook and leave the biases and the normalization parameters in FP32, the step sizes are initialized by a grid search on the mean squared error, and the latent weights are optimized by SGD while the positive step sizes use a separate Adam optimizer. Every forward pass uses the fake-quantized weights, the latent weights are restored before the optimizers take their step, and the inner solver is warm-started from the previous call and runs once every four outer steps. The learning rate of the weights follows a cosine decay down to an explicit floor, and where it is enabled the Adam rate of the step sizes follows the same multiplier, so the grid stops moving once the weights have. Four architectures are distilled from a frozen FP32 copy of their own checkpoint, the two transformers at $\alpha = 0.9$ and AlexNet and EfficientNet-B0 at $\alpha = 0.5$, while neither ResNet is, because their LSQ controls already exceed the FP32 accuracy and a strong teacher anchor pulls them back down. Batch-normalization statistics are recalibrated before every evaluation on the convolutional models. Table 2 gives every setting per run and Table 3 the quantized fraction of each network, while Appendix E separates the parts of this recipe that are held identical across architectures from the parts that are not.

A.2 CONFIGURATIONS, CHECKPOINTS AND SERIALIZATION

Table 2: Configuration of every run reported in the paper. Rows within a block differ only in the coefficient triple and, where stated, the budget. “rel.” is the step-size rate expressed as a fraction of each tensor’s own initial step; “abs.” is a single absolute value.

Network	Role	Ep.	Batch	Weight LR, floor	Step-size LR	Distill.	(T_1, T_2, T_3)
ResNet-18	control	60	1024	10^{-2} , 1%	10^{-5} abs.	no	(0, 0, 0)
	ablation, 3 arms	60	1024	10^{-2} , 1%	10^{-5} abs.	no	see Appendix A.3
	headline, 3 repeats	60	1024	10^{-2} , 1%	10^{-5} abs.	no	$(10^{-5}, 10^{-7}, 6 \cdot 10^{-8})$
	frontier, 2 points	100	1024	10^{-2} , 1%	10^{-5} abs.	no	$T_3 \in \{10^{-7}, 2.6 \cdot 10^{-7}\}$
ResNet-50	control	20	1024	10^{-2} , 1%	10^{-5} abs.	no	(0, 0, 0)
	frontier, 2 points	20	1024	10^{-2} , 1%	10^{-5} abs.	no	$T_3 \in \{3 \cdot 10^{-8}, 10^{-7}\}$
DeiT-Small	control	25	2048	10^{-4} , 5%	10^{-5} abs.	0.9	(0, 0, 0)
	frontier, 4 points	25	2048	10^{-4} , 5%	10^{-5} abs.	0.9	ray, $\alpha \in \{\frac{1}{5}, \frac{1}{2}, 1, 2\}$
ViT-B/16	control	20	1024	$5 \cdot 10^{-5}$, 5%	10^{-5} abs.	0.9	(0, 0, 0)
	frontier, 2 points	20	1024	$5 \cdot 10^{-5}$, 5%	10^{-5} abs.	0.9	ray, $\alpha \in \{\frac{1}{5}, 1\}$
AlexNet	control	60	1024	$2 \cdot 10^{-3}$, 1%	10^{-3} rel.	0.5	$(10^{-5}, 0, 0)$
	ablation, 3 arms	60	1024	$2 \cdot 10^{-3}$, 1%	10^{-3} rel.	0.5	see Appendix A.3
EfficientNet-B0	control	30	4096	$2 \cdot 10^{-2}$, 5%	10^{-3} rel.	0.5	(0, 0, 0)
	PRESTO	30	4096	$2 \cdot 10^{-2}$, 5%	10^{-3} rel.	0.5	$(10^{-5}, 2.1 \cdot 10^{-7}, 6 \cdot 10^{-8})$

The reference dose of the transformer ray is $(T_2, T_3) = (5 \cdot 10^{-8}, 1.5 \cdot 10^{-8})$, and α scales both. Two epoch counts warrant a word: ResNet-50 and ViT-B/16 run for twenty epochs against ResNet-18's sixty because their per-epoch cost is five to ten times higher, and every one of those runs ended with both axes still improving, so the shorter budgets understate rather than flatter them. EfficientNet-B0 additionally gives eight bits to its sixteen depthwise tensors and its input convolution, 3.7% of its weights, counted at full cost in every ratio; AlexNet gives eight bits to its two input convolutions, 0.07% of its weights.

Table 3 records which part of each checkpoint the packet covers, since every ratio in the paper is converted through $\text{model} = rf + (1 - f)$. The quantized set is every convolutional and linear weight tensor; what stays in floating point is the biases, the normalization vectors, the class token and the positional embeddings. On ViT-B/16 the set needs one remark, because torchvision stores the packed query, key and value projection of each block in `in_proj_weight`, a parameter that is not a module weight: quantizing only the tensors that look like module weights would cover 75.15% of that network instead of 99.68% and would make every ratio incomparable with the published ones.

Table 3: Quantized fraction f of each checkpoint, the number of tensors it covers, and the residue $1 - f$ that stays in floating point and is charged at full cost in every ratio we report.

Network	Tensors	Parameters	f
ResNet-18	21	11,678,912	99.909%
ResNet-50	54	25,502,912	99.788%
DeiT-Small	50	21,912,576	99.374%
ViT-B/16	50	86,292,480	99.682%
AlexNet	8	61,090,496	99.983%
EfficientNet-B0	82	5,236,192	99.010%

Reproducibility. The symbolic coefficients above follow the notation of Section 2.2: T_1 is the perspective coefficient, T_2 the sparsity coefficient, T_3 the entropy coefficient. Archived logs predating that renaming print the last two in the opposite positions, and the values reported here have been mapped to the notation of this paper. Every run starts from a public checkpoint of ResNet-18 or ResNet-50 (He et al., 2016), AlexNet (Krizhevsky et al., 2012), DeiT-Small (Touvron et al., 2021), ViT-B/16 (Dosovitskiy et al., 2021) or EfficientNet-B0 (Tan and Le, 2019), identified by the content hash carried in its file name:

Network	Checkpoint
ResNet-18	resnet18-f37072fd
ResNet-50	resnet50-0676ba61
DeiT-Small	deit_small_patch16_224-cd65a155
ViT-B/16	vit_b_16-c867db91
AlexNet	alexnet-owt-7be5be79
EfficientNet-B0	efficientnet_b0_rwightman-7f5810bc

The serialized package is produced by one fixed command for every run and every architecture: the support is a packed bit mask, one bit per weight; the values are the integer symbol indices of the non-zero weights, packed without padding; the metadata is one fp32 step size per tensor, or one per output channel where per-channel grids are used. All three streams are compressed with `zstd` at level 22 with multithreading enabled, which is the only lossless coder used anywhere in this paper and is applied identically to our runs and to any baseline we re-serialize. Batch-normalization statistics, where the architecture has them, are recalibrated over 50 local batches per rank before every validation pass. The dual is updated every four optimizer steps with $N_{\text{dual}} = 3$ iterations, except on the two ViT-B/16 frontier points where it is 6, and the LSQ step sizes use their own optimizer and learning rate, reported per run in Table 2. Two quantities of the dual are fixed for every run and are not tuned: the lower clamp on the relaxed counts is $c_{\min} = 10^{-2}$, which fixes the interval onto which the multiplier is projected, derived in Appendix C.8, and the multipliers of a tensor are initialized uniformly at random inside that interval and then warm-started from one call to the next. Training runs under PyTorch with `torchvision`, `timm` ($\geq 0.9.12$), `webdataset` and `zstandard`, on sixteen A100-SXM-64GB across four nodes under DDP, with the ImageNet shards read through `webdataset`.

We state one limitation of this protocol explicitly. The training script sets no global random seed. The three repeated ResNet-18 runs of Section 4.2 are therefore repeats of a bit-identical command rather than a seed sweep, and the spread we quote, 0.083 points of accuracy and 0.015 of packet, measures the run-to-run nondeterminism of data order under DDP and of non-deterministic GPU reductions. That is the noise floor a reader should compare our differences against, and it is the quantity we use for that purpose throughout; it is not a measurement of sensitivity to initialization, which we do not claim.

Table 4: Additional architectures, on the same whole-checkpoint accounting as Table 1. These are superseded recipes kept as evidence of transfer: the “LSQ” rows carry no sparsity and no entropy term, the AlexNet one keeping the ridge at 10^{-5} , and both networks use per-channel step sizes here, which Appendix B.7 shows is paid for twice. The runs of Tables 1–6 supersede every row.

Architecture	Variant	Epochs	Top-1	Δ FP32	Size	Sparsity
AlexNet	LSQ + ridge	60	56.552	+0.028	10.78%	16.11%
AlexNet	PRESTO, T_2 only	60	54.938	−1.586	3.68%	70.60%
EfficientNet-B0	LSQ	10	76.590	−0.976	12.43%	16.19%
EfficientNet-B0	PRESTO, T_3 only	10	76.452	−1.114	12.07%	18.06%
EfficientNet-B0	PRESTO, T_2 only	20	74.746	−2.820	9.06%	39.83%

A.3 TERM ABLATIONS

Two ablations, one per architecture family. Within each, the runs share every flag but the coefficient triple, the budget and the checkpoint included.

Table 5: Term ablation. ResNet-18 at sixty epochs against 69.734; AlexNet at sixty epochs against 56.524. *Rate* is points of size bought per point of accuracy spent, measured from the LSQ control of the block: the (0, 0, 0) arm on ResNet-18, the ridge-only arm on AlexNet, which is that block’s control. *H* is the entropy of the non-zero symbols at the end of the run. The rates are secants to a concave frontier between arms that do not end at the same size, and Section 4.3 explains why we do not rank the terms by them. The sparsity each arm reaches is quoted in Section 4.3.

Network	(T_1, T_2, T_3)	Top-1	Size	Rate	<i>H</i>
ResNet-18	(0, 0, 0)	70.490	9.84%	–	3.215
	$(10^{-5}, 0, 0)$	70.392	9.87%	–	3.210
	$(10^{-5}, 10^{-7}, 0)$	70.312	6.96%	16.17	2.526
	$(10^{-5}, 0, 6 \cdot 10^{-8})$	70.090	5.03%	12.04	1.926
	full	70.020	4.89%	10.55	1.902
AlexNet	$(10^{-5}, 0, 0)$	56.500	10.58%	–	3.334
	$(10^{-5}, 3 \cdot 10^{-7}, 0)$	55.770	3.04%	10.33	1.688
	$(10^{-5}, 0, 2.3 \cdot 10^{-8})$	55.882	3.48%	11.49	1.721
	full	55.644	2.85%	9.02	1.686

The AlexNet entropy dose was not transferred. Carrying ResNet-18’s 6×10^{-8} across put the arm far past its operating point, at 2.55% of the checkpoint but 1.96 points below it, and the resulting exchange rate, 4.15, would have made the entropy term look far worse than it is. That run traced the dose–response curve one epoch at a time and the corrected dose, 2.3×10^{-8} , was read off it by the exposure rule of Appendix D. It is a concrete instance of the general point made there: an exchange rate measured at a transferred dose is not a property of the term.

On AlexNet the full method reaches 2.85% of the checkpoint at 0.880 below the full-precision accuracy. Deep Compression (Han et al., 2016) reports 2.86% at parity with its own baseline, so at the same size we are less accurate and we say so; AlexNet is a network whose parameters sit overwhelmingly in three dense layers, where a pipeline of magnitude pruning, iterative retraining, *k*-means weight sharing and Huffman coding is at its strongest, and we report it as a transfer architecture rather than a competitive one.

Table 6: Additional rows: published points that Table 1 does not need in order to place our frontier, and the intermediate doses of the two transformer rays. Same conventions as Table 1.

Network	Variant	Ep.	Top-1	Size	Spars.
ResNet-18	PRESTO (10^{-5} , 10^{-7} , 10^{-7})	60	69.606	4.23%	68.6%
ResNet-18	PRESTO (10^{-5} , 10^{-7} , $2.6 \cdot 10^{-7}$)	100	68.330	3.02%	80.6%
ResNet-50	PRESTO (10^{-5} , 10^{-7} , 10^{-7})	20	75.444	3.72%	75.3%
ResNet-18	Metz et al. (Metz et al., 2024)	–	69.0 [69.8]	3.57%	–
ResNet-18	LilNetX [†] (Girish et al., 2023)	–	68.8 [70.0]	2.16%	58.3%
ResNet-50	DeepCABAC (Wiedemann et al., 2019)	–	74.51 [76.13]	10.14%	–
ViT-B/16	NNCodec (Becking et al., 2023)	–	80.41 [81.07]	10.44%	–
EfficientNet-B0	NNCodec (Becking et al., 2023)	–	76.29 [77.67]	10.08%	–
ViT-B/16	PRESTO, one fifth of the dose	20	80.682	7.14%	55.7%
DeiT-Small	PRESTO, half dose	25	79.704	10.63%	30.9%
DeiT-Small	PRESTO, one fifth of the dose	25	79.694	11.98%	21.5%
DeiT-Small	Q-ViT 4-bit (Li et al., 2022)	300	80.9 [79.9]	12.93%*	–

A.4 MASK COST AND SPARSITY

The percentages in this subsection are shares of the FP32 bytes of the quantized tensors, the accounting in which the packet decomposes additively; over the whole checkpoint each of them is larger by $1 - f$. The support mask is compressed with the same coder as everything else, and the i.i.d. Bernoulli model $H(p)/32$ of FP32 storage is a first-order description of its cost rather than the cost itself: at 51.53% sparsity the model gives 3.12% and the serializer produces 2.905%, because the mask is not memoryless. What the model does capture is the shape, since it is maximized at $p = 0.5$. Past that point extra zeros are rewarded twice, by a shorter value stream and by a cheaper mask, and our runs show it: at 51.53% sparsity the mask costs 2.905%, at 67.37% it costs 2.608% and at 68.55% it costs 2.563%. A method that stops at half sparsity sits exactly on the worst point of that curve.

A.5 NOTES ON THE TRANSFER ARCHITECTURES

Table 4 in Appendix A.2 summarizes AlexNet and EfficientNet-B0, which are less tuned and serve as evidence of transfer. On AlexNet, sweeping T_2 traces a dose-response curve; on the recipe of this paper the exchange rate is still 10.3 points of size per accuracy point at 78% sparsity, against a knee between 42% and 51% on the earlier per-channel recipe, so the knee is a property of the procedure and not of the network. Section 4.4 reports where we stand there and why. EfficientNet-B0 is the opposite case, with its knee at 23%: a network designed for accuracy per parameter has less redundancy to remove.

Table 7: Change from the first to the last epoch, EfficientNet-B0. “Step size” is the median over tensors of the ratio between the final and initial LSQ step size; “latent p_1 ” is the relative change of the first percentile of the latent weights; “entropy” is the entropy of the nonzero symbols in bits per surviving weight.

Run	Step size	Latent p_1	Mask	Values	Entropy
Control (0, 0, 0)	$\times 0.980$	-3.07%	$+0.037$	-0.152	-0.036
T_2 only	$\times 1.359$	-6.99%	$+1.049$	-4.558	-1.177
T_3 only	$\times 0.988$	-3.23%	$+0.172$	-0.649	-0.156

A.6 HARDWARE AND WALL-CLOCK COST

All runs use four nodes of the Leonardo supercomputer, partition `boost_usr_prod`, each node carrying four NVIDIA A100-SXM-64GB GPUs and 32 CPU cores per task, for sixteen GPUs in total. Data are read from WebDataset shards. Table 8 reports the per-epoch training time excluding the evaluation and serialization pass, as mean and standard deviation over all epochs after the first. The first epoch is listed separately because the entropy warm-up leaves the dual solver switched off in it, which makes it a free measurement of the same recipe without the regularizer’s dominant cost.

Two costs are separable in that table. The perspective ridge and the sparsity term go through a per-step closed-form path and are free: on ResNet-18 the ridge-only and no-entropy runs cost 108 seconds per epoch against 108 for plain LSQ. The entropy term is the expensive one, because it calls the dual solver once every four optimizer steps: the same network costs 447 seconds per epoch with

Table 8: Per-epoch training time, excluding metrics. Sixteen A100 GPUs; the global batch is the one reported for each architecture in Table 2.

Run	Epochs	Time per epoch (s)	First epoch (s)
ResNet-18, LSQ only	60	108 ± 1	109
ResNet-18, ridge only	60	108 ± 0	108
ResNet-18, no entropy	60	108 ± 1	108
ResNet-18, no sparsity	60	427 ± 31	110
ResNet-18, full PRESTO	60	447 ± 33	108
ResNet-50, LSQ only	20	109 ± 0	109
ResNet-50, full PRESTO	20	997 ± 107	157
ResNet-50, $T_3 = 10^{-7}$	20	920 ± 50	157
ResNet-18, full PRESTO	100	474 ± 36	139
DeiT-Small, LSQ + distillation	25	133 ± 1	133
DeiT-Small, full PRESTO	25	443 ± 6	132
EfficientNet-B0, LSQ + distillation	30	147 ± 1	148
EfficientNet-B0, full PRESTO	30	384 ± 9	147
ViT-B/16, LSQ + distillation	20	218 ± 1	218
ViT-B/16, full PRESTO, $N_{\text{dual}} = 3$	20	1213 ± 22	275
ViT-B/16, full PRESTO, $N_{\text{dual}} = 6$	20	2229 ± 45	279

it, a factor of 4.1. The same separation holds on AlexNet, whose ridge and sparsity arms run at 108 seconds per epoch and whose entropy arms run at 682, a factor of 6.3; those arms are not listed in Table 8 because we logged their mean without a per-epoch spread.

That factor is not a property of the network size alone, and comparing networks by it requires holding N_{dual} fixed, since the two ViT-B/16 rows of Table 8 differ in exactly that. Read at $N_{\text{dual}} = 3$ throughout, dividing the entropy overhead by the 313 dual calls of each epoch makes one call cost 1.08 seconds on ResNet-18, 2.84 on ResNet-50 and 3.18 on ViT-B/16. Per tensor those are 51.6, 52.5 and 63.6 milliseconds, nearly flat, whereas per million quantized weights they are 93, 111 and 37, a factor of three apart: ViT-B/16 carries 3.4 times the weights of ResNet-50 across four fewer tensors and costs only 12% more per call. The solver builds one lower hull and runs one FISTA loop per tensor, and over the whole range we measured it is dominated by that per-tensor overhead rather than by the per-weight sweep, a least-squares fit giving 48 milliseconds per tensor and 9 nanoseconds per weight, with the per-tensor term accounting for between 76 and 92 per cent of a call. At $N_{\text{dual}} = 6$ the same ViT-B/16 configuration costs 6.43 seconds per call, almost exactly twice as much, which is what the linear dependence on N_{dual} predicts and which is why the two settings cannot be put on one scaling argument. Batching the per-tensor work is the obvious optimization and we have not attempted it.

The overhead in context, and the one run that uses a different solver setting. Against its own control the entropy term costs $3.3\times$ on DeiT-Small, $2.6\times$ on EfficientNet-B0, $4.1\times$ on ResNet-18, $5.6\times$ on ViT-B/16 at $N_{\text{dual}} = 3$ and $9.1\times$ on ResNet-50. The two ViT-B/16 frontier points are the only runs in the paper at $N_{\text{dual}} = 6$, which doubles the dual convergence rate and raises the overhead to $10.2\times$. We ran the headline ViT-B/16 operating point at both settings: $N_{\text{dual}} = 6$ gives 80.372 at 4.41% of the checkpoint and $N_{\text{dual}} = 3$ gives 80.260 at 4.44%. The difference, 0.112 accuracy points, is 1.3 standard deviations of our seed study and favours the larger value, which is why the ViT-B/16 rows use it; but both settings clear the standardized codec at that size by more than five accuracy points, so the ViT-B/16 claim does not depend on the choice.

A.7 LEARNING AND COMPRESSION CURVES

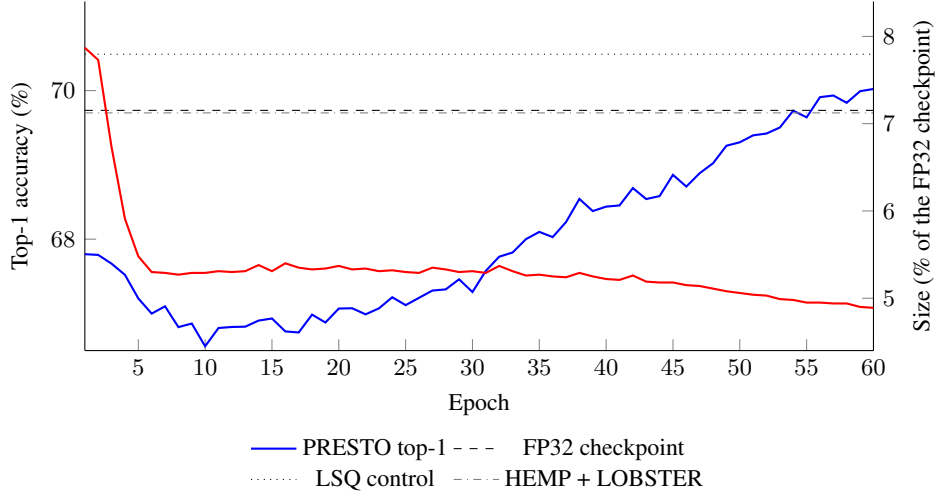


Figure 1: ResNet-18, the headline configuration $(T_1, T_2, T_3) = (10^{-5}, 10^{-7}, 6 \times 10^{-8})$ over sixty epochs. Blue, left axis: top-1 accuracy. Red, right axis: the serialized packet as a percentage of the FP32 checkpoint. The two phases are visible: the compression is delivered early, in the first ten epochs, while the accuracy recovers over the whole cosine and crosses the full-precision checkpoint at epoch fifty-four. The curve is one of the three repeats; it ends at 4.89% of the checkpoint, which is also the mean of the three to the second decimal. Reference lines are the FP32 checkpoint, the LSQ control at the same budget, and the closest published point.

B FURTHER EXPERIMENTAL RESULTS

This appendix reports the experiments that support Section 4 without being needed in order to follow it. Nothing here has been summarized, so every number, every table and every caveat is given in full. Figure 2 places every run of the paper and every published point within its range on a single panel.

B.1 QUANTIZATION-AWARE TRAINING ALONE MAKES MODELS LARGER

One measurement motivates the method before any comparison is made. Table 9 takes the LSQ-only control of every architecture and reads its packet at the first epoch against its packet at the end, together with the entropy of its non-zero symbols over the same interval.

Table 9: Quantization-aware training on its own, with no regularizer, makes the quantized model *harder* to compress while it gains accuracy. Four architectures, five controls, no exception. Sizes here are the packet over the FP32 bytes of the *quantized tensors*, the accounting in which the relative change is read; over the whole checkpoint each row is larger by $1 - f$, so the last ResNet-18 control reads 9.84% in Table 1.

Control	Network, epochs	Packet, first $\rightarrow$ last	Change	$H(\text{non-zero})$
LSQ only	ResNet-18, 20	8.63% $\rightarrow$ 9.34%	+8.2%	2.796 $\rightarrow$ 3.080
LSQ only	ResNet-18, 60	8.59% $\rightarrow$ 9.76%	+13.6%	2.788 $\rightarrow$ 3.215
LSQ only	ResNet-50, 20	8.83% $\rightarrow$ 9.78%	+10.8%	2.854 $\rightarrow$ 3.147
LSQ + distill.	DeiT-Small, 25	10.84% $\rightarrow$ 11.44%	+5.5%	3.352 $\rightarrow$ 3.541
LSQ + distill.	ViT-B/16, 20	9.67% $\rightarrow$ 10.19%	+5.4%	3.169 $\rightarrow$ 3.316

Sixty epochs of four-bit quantization-aware training on ResNet-18 gain 0.6 accuracy points and at the same time make the serialized model 13.6% larger, while the entropy of the symbol histogram rises in every control that we ran, so a low bit width clearly does not imply a low entropy and training left to itself drifts in the wrong direction. Under the regularizer the same quantity falls by between 0.468 and 1.415 bits across our runs, so PRESTO does not improve an existing trend but reverses it.

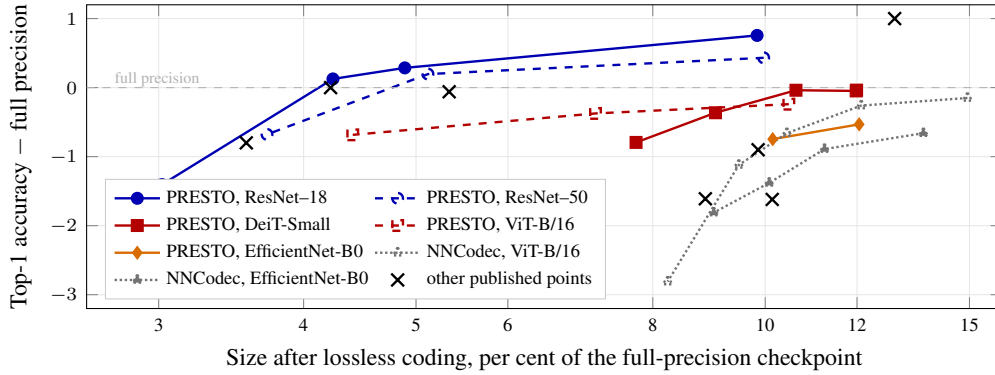


Figure 2: Accuracy against size after lossless coding, as reported by each method for the checkpoint it uses. Accuracy is the difference from each method’s own full-precision measurement, the only way to put five architectures on one panel; the raw accuracies behind every head-to-head claim are in Table 1. Crosses are single published points; LilNetX, at 2.16% and -1.2 , falls to the left of the panel.

B.2 BUDGET AGAINST DOSE ON RESNET-18

Budget, rather than dose, improves both axes at once. At $T_3 = 10^{-7}$ and sixty epochs the run lands 0.128 below the checkpoint, while the same configuration at one hundred epochs reaches 0.126 above it at the same packet and was still gaining 0.03 points per epoch when the schedule ended. Raising or lowering T_3 trades one axis against the other, whereas lengthening the schedule does not. The frontier does bend, and we report where: the marginal cost per point of size is 0.625 accuracy points between the sixty-epoch rows of Tables 1 and 6 and 1.255 between the two hundred-epoch ones, the deeper of which is at 3.02% and 1.404 below.

B.3 JOINT OPTIMIZATION BEATS THE SAME OBJECTIVE DECOMPOSED

PRESTO optimizes sparsity and code length together, whereas the obvious alternative is to optimize them in sequence as the classical pipeline does, and the question is whether the joint formulation earns the complexity that it adds. We answer that question by decomposing our own objective rather than by comparing against a different method, so that nothing except the ordering changes.

Two pipelines run twenty epochs on one term and then twenty on the other, in both orders, starting from the same checkpoint and using the same coefficients as the joint run. Each stage is a complete twenty-epoch cosine, identical in shape to the joint reference, so the schedules are comparable stage by stage. The second stage reloads the latent weights and re-initialises the step sizes by MSE, which is what a sequential method does and is also, as we will see, part of the mechanism. The only confound that remains is the budget, and it favours the pipeline, which receives forty epochs against the twenty of the joint run, so we also report the joint run at forty epochs, where the two are budget-matched.

Table 10: Decomposing the objective, ResNet-18, $(T_1, T_2, T_3) = (10^{-5}, 10^{-7}, 3 \cdot 10^{-8})$ throughout. At matched budget the three accuracies agree to within 0.034, well inside the 0.083 spread between repeats, so the comparison is purely on size.

Optimization	Epochs	Top-1	Size
Joint	20	69.668	5.83%
Joint	40	70.080	5.82%
Sparsity alone, first stage	20	70.098	7.46%
Sparsity then entropy	20 + 20	70.114	6.08%
Entropy alone, first stage	20	69.782	6.09%
Entropy then sparsity	20 + 20	70.108	7.21%

At forty epochs the joint run is 4.3% smaller than the better pipeline and 19.4% smaller than the worse one at the same accuracy, and the sharper way of reading Table 10 is to ask what the second

twenty epochs produce. Against the joint run at twenty epochs, which is the common reference for all three, every configuration ends between 0.41 and 0.45 accuracy points higher, while the joint run holds its size to within 0.01 points and the two pipelines end 0.25 and 1.39 points larger. Read instead against each pipeline’s own first stage, which the table now reports, the asymmetry is starker: the entropy stage of the first pipeline converts its twenty epochs into 1.38 points of size for 0.016 of accuracy, whereas the sparsity stage of the second gives back 1.12 points of size for 0.326 of accuracy. Sequential optimization therefore spends the extra budget on accuracy that it can no longer spend on size.

The mechanism behind that behaviour is visible in the entropy of the non-zero symbols. Re-quantizing between the two stages costs 0.343 bits in one order and 0.445 bits in the other, because the grid is re-fitted to the latents that were loaded and the symbols are remapped, and the entropy stage recovers that cost and more, at -0.742 bits over its twenty epochs, whereas the sparsity stage cannot recover it and adds a further $+0.040$. Running the entropy term first is therefore self-defeating, because it gains 0.759 bits, the re-fit gives back 0.445 of them and the sparsity stage then finishes 0.274 bits above the point at which the entropy stage began. Ordering matters, in the direction that the classical pipeline happens to get right, and both orders lose to solving the two problems together.

B.4 ACCURACY-SIZE FRONTIER ON VISION TRANSFORMERS

On both transformers the LSQ control does not clear the checkpoint the way the ResNet controls do, so the object we report is the frontier against a budget-matched control rather than a lossless claim. On ViT-B/16 five control runs spanning learning rates from 3×10^{-5} to 10^{-4} and budgets from ten to thirty epochs land within a tenth of a point of each other and about two tenths below full precision, from -0.142 to -0.242 : there is no headroom to spend, and no amount of budget creates any. Those five predate the packed-attention fix of Appendix A.2, so only their accuracies are used here and their sizes are not reported.

Against that control PRESTO removes 58% of the ViT-B/16 packet for 0.444 accuracy points, reaching 4.41% of the checkpoint at 72.9% sparsity, the highest of any run here. On DeiT-Small the curve is traced by four doses along the ray and the marginal cost rises monotonically: 0.001, 0.208 and 0.325 accuracy points per point of packet, measured from the control through the half, reference and double doses. The first of those is the interesting one, because between the control and half the dose the packet falls from 11.99% to 10.63% while the accuracy moves by 0.002 points, which is far inside the noise, so on this network the first 11% of compression is free and the trade-off only begins beyond that point.

A fifth dose deserves a separate mention, the fifth of the reference reported in Table 6, because there the sparsity rises from 13.7% to 21.5% and the packet does not move at all, from 11.99% to 11.98%. Below 50% sparsity the support mask costs about $H(p)/32$ per weight and grows faster than the symbol stream shrinks, so any sparsity gained there is paid for twice, and the same arithmetic explains why our most compressed ResNet-18 rows carry a cheaper mask than less sparse ones once they pass $p = 1/2$.

B.5 THE TWO TERMS ACT THROUGH DIFFERENT MECHANISMS

The sparsity and entropy terms are not two dials on a single mechanism. Measured on the five matched ResNet-18 ablation arms of Appendix A.3, the sparsity term contracts the latent distribution furthest, since its first percentile moves 14.31% towards zero against 11.66% for the ridge alone, and on EfficientNet-B0, where we also logged the step sizes (Table 7), it inflates them by 36% while the control leaves them unchanged, so widening the interval that maps to zero is the way in which it creates sparsity. The entropy term leaves both of those quantities where the ridge alone leaves them, at 11.25% and $\times 0.988$, and works on the assignment instead, cutting the value bytes by 65% against the sparsity term’s 24% and the entropy of the non-zero symbols by 0.858 bits, while the sparsity term moves that entropy by only 0.024 and the control lets it rise by 0.427. The arms do not start from a common value, 2.502 bits under the sparsity term against 2.78 elsewhere, because the first measurement is taken at the end of the first epoch and the closed-form terms act from the first optimizer step while the entropy term is still in its warm-up; the quantity to compare is therefore the change along each run and not its endpoints.

The separation is therefore not pruning against coding. Both arms create zeros, and on ResNet-18 the entropy term creates more of them, 60.30% sparsity against 45.56%, which follows from its minimizer at $c_b = 1/e$ pushing mass into the excluded zero symbol. What only one of the two does is shorten the code of the weights that survive, and Section 2.2 explains why we report that as a measurement rather than as a property of the term.

Why they do not add. Whichever term arrives first, the other has little left to remove, and both ablations show it on the same quantity. On AlexNet the two arms reach the same symbol distribution by different routes: the entropy of the non-zero symbols starts at 2.892 bits under the sparsity term and 3.326 under the entropy term, and *both drive it to the same floor*, 1.688 and 1.721 bits, with the combination ending at 1.686, which is indistinguishable from sparsity alone. On ResNet-18 the floor is not shared, because the sparsity term stops at 2.526 bits while the entropy term reaches 1.926, and there the combination ends at 1.902, which is indistinguishable from entropy alone. In both blocks the full objective lands on the floor of whichever single term went lower, and the second term then contributes the few per cent of size reported in Section 4.3 rather than a second reduction. Which term gets there first is the architecture-dependent quantity, and it is the same ordering as the shares in Section 4.3.

B.6 THE ALEXNET DEFICIT: WHAT IT IS NOT, AND WHAT WE THINK IT IS

It is not the budget, since AlexNet has converged at sixty epochs, the last ten gaining 0.010 points per epoch on the control and losing 0.004 on the entropy arm. It is also not a healing phase too short to recover from the pruning, the natural suspicion given how Deep Compression is trained, because our sparse structure is 95% final by epoch 17 to 20 and leaves twenty-eight to thirty-one epochs above a tenth of the base rate, against two on ResNet-18, where it settles only at epoch 46. AlexNet therefore gets an order of magnitude more high-rate retraining on its final structure than the network that finishes above full precision, and the full objective still ends 0.880 below the checkpoint.

What we believe it is instead is allocation, and the arithmetic is worth writing down even though we did not run the arm that would test it. Our sparsity is not allocated: one coefficient pushes in the same way on eight tensors whose sizes span three orders of magnitude, while both methods ahead of us assign a rate per layer. The five convolutions hold 4.04% of the weights, so pruning them at 80% gives 0.218 points of packet, worth 0.021 accuracy points at the exchange rate we measure here and repaid by two extra points of sparsity on the dense layers: we spend the feature extractor to save a fifth of a point that the dense layers would return for almost nothing. This is a hypothesis and not a measurement, and a per-layer dose is left to future work.

B.7 PER-CHANNEL QUANTIZATION IS PAID FOR TWICE

Per-channel step sizes are usually treated as accuracy that comes for free. On EfficientNet-B0, switching a matched control from per-channel to per-tensor cost 0.098 accuracy points and returned 1.03 points of packet, which is a rate as favourable as the one the sparsity term gives at its knee, and only 0.597 of that return was the metadata, a hard floor of one fp32 number per output channel over 82 tensors. The remaining 0.345 came from the value stream, because one shared grid per tensor gives the coder a single symbol distribution instead of one distribution per output channel and the pooled stream is more concentrated. Per-channel quantization therefore costs code length as well as metadata, and on networks with many small tensors the two together are worth more than the accuracy that it provides.

B.8 NEGATIVE RESULTS

Four failures shaped the method and we report them because each changed what we believed.

The entropy coefficient is only as strong as its dual is converged. T_3 reaches the weights through one channel, the multiplier, and the dual range to be crossed is proportional to T_3 while an absolute dual step is not. Raising T_3 tenfold therefore left the applied gradient identical to four significant figures and compressed *less*, not more. The fix is to measure the step as a fraction of the dual range; Appendix C.9 derives the rate. The general lesson is that when a coefficient looks inert, the dual

variable through which it acts should be checked for convergence before the coefficient itself is touched.

The dose cannot be derived, only measured and transported. Three analytic models for the sparsity coefficient failed in sequence, the first inert by three orders of magnitude, the second divergent because it had been calibrated on the wrong branch of the inner asymptotics, and the third overdosed tenfold because the dominant channel was not present in the equation. What transfers between runs is the cumulative exposure, the coefficient times the schedule sum, never the coefficient itself; a ramp row reports the exposure accumulated up to that epoch and reproducing it at fixed dose requires redoing the integral, a factor of 4.4 in one case.

A stopping criterion we wrote would have given the wrong answer. We had committed to abandoning the entropy channel on EfficientNet-B0 if it did not separate from the control within four epochs at thirty times the reference dose. It did not separate, but the cause was the dual rather than the dose, and the channel opened as soon as the step was made relative.

One diagnostic is worthless and we stopped using it. The accuracy of the network evaluated at its latent weights collapses by up to seventeen points without predicting anything about the deployed model, because the straight-through estimator applies a gradient computed at the quantized point to a latent that has no restoring force toward it. The gap scales with the step size, not with the regularizer, and it does not appear in this paper.

C SUPPLEMENTARY DERIVATIONS

C.1 LEARNED-STEP QUANTIZATION: FULL STATEMENT

Let $Q_n < 0 < Q_p$ be two integers denoting the minimum and maximum integer codes, respectively. The integer codebook is

$$\mathcal{Q} := \{q \in \mathbb{Z} : Q_n \leq q \leq Q_p\}, \quad C := |\mathcal{Q}| = Q_p - Q_n + 1,$$

where $\mathbb{Z}$ is the set of integers and C is the number of symbols. We associate each tensor ℓ with a quantization step size $a_\ell \in \mathbb{R}_{>0}$. The collection of step sizes is $a = \{a_\ell\}_{\ell \in \Lambda}$.

For $\alpha, \beta, u \in \mathbb{R}$ with $\alpha \leq \beta$, define

$$\text{clip}(u; \alpha, \beta) := \min\{\beta, \max\{\alpha, u\}\}.$$

Moreover, let $\text{round}(u)$ denote the integer nearest to u , with ties resolved toward the even integer.

For weight $w_i^{(\ell)}$, we set

$$r_i^{(\ell)} := \frac{w_i^{(\ell)}}{a_\ell}, \tag{13}$$

$$\bar{w}_i^{(\ell)}(a_\ell) := a_\ell \text{clip}\left(r_i^{(\ell)}; Q_n, Q_p\right), \tag{14}$$

$$\hat{w}_i^{(\ell)} := a_\ell \text{round}\left(\frac{\bar{w}_i^{(\ell)}(a_\ell)}{a_\ell}\right). \tag{15}$$

$r_i^{(\ell)}$ is the weight normalized by the step size. The clipped latent weight $\bar{w}_i^{(\ell)}(a_\ell)$ remains real-valued but is restricted to the representable interval $[a_\ell Q_n, a_\ell Q_p]$. Rounding its normalized value then produces $\hat{w}_i^{(\ell)}$, which belongs to the real-valued codebook $a_\ell \mathcal{Q}$ and is used in the forward pass. The quantized weights of the entire network are $\hat{w}(a) = \{\hat{w}^{(\ell)}\}_{\ell \in \Lambda}$.

PRESTO uses Learned Step Size Quantization (LSQ) (Esser et al., 2020) to propagate a surrogate gradient through the rounding operation. The PRESTO inner problem operates on $\bar{w}_i^{(\ell)}$, ensuring that the quantizer and the regularizer share the same representable domain.

C.2 DERIVATION OF THE DECOMPOSED DUAL

Fix a tensor $\ell \in \Lambda$, a clipped latent-weight vector $\bar{w}^{(\ell)}$, and a step size a_ℓ . For each count constraint, let $\xi_b^{(\ell)} \in \mathbb{R}$ be an unrestricted Lagrange multiplier, and let $\xi^{(\ell)} = (\xi_b^{(\ell)})_{b \in \mathcal{B}}$ collect these multipliers.

For the Lagrangian relaxation of each count constraint, we use the sign convention

$$\xi_b^{(\ell)} \left(\sum_{i \in \mathcal{I}_\ell} x_{i,b}^{(\ell)} - c_b^{(\ell)} \right).$$

The partial Lagrangian of tensor ℓ , obtained by dualizing only the count constraints, is

$$\begin{aligned} \mathcal{L}_\ell(x^{(\ell)}, y^{(\ell)}, c^{(\ell)}; \xi^{(\ell)}) &:= T_1 \sum_{i \in \mathcal{I}_\ell} \frac{(\bar{w}_i^{(\ell)})^2}{y_i^{(\ell)}} + T_2 \sum_{i \in \mathcal{I}_\ell} y_i^{(\ell)} \\ &\quad + T_3 \sum_{b \in \mathcal{B}} c_b^{(\ell)} \log_2 c_b^{(\ell)} + \sum_{b \in \mathcal{B}} \xi_b^{(\ell)} \left(\sum_{i \in \mathcal{I}_\ell} x_{i,b}^{(\ell)} - c_b^{(\ell)} \right). \end{aligned} \quad (16)$$

The variables $x^{(\ell)}$ and $y^{(\ell)}$ retain the domain constraints in Eq. (7) and the representation and mass constraints in Eqs. (4) and (5), while $c_b^{(\ell)} \geq 0$ for every $b \in \mathcal{B}$. The count constraints themselves are no longer imposed explicitly, because their violation is priced by $\xi^{(\ell)}$.

Let $\varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}(\xi^{(\ell)})$ denote the minimum of $\mathcal{L}_\ell$ over these remaining feasible variables. The function $\varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}$ is the Lagrangian dual function and is concave in $\xi^{(\ell)}$, because it is the pointwise infimum of affine functions of the multipliers. Rearranging Eq. (16) yields

$$\begin{aligned} \varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}(\xi^{(\ell)}) &:= \min_{\substack{(x_i^{(\ell)}, y_i^{(\ell)}) \in \mathcal{F}_i^{(\ell)} \forall i \in \mathcal{I}_\ell, \\ c_b^{(\ell)} \geq 0 \forall b \in \mathcal{B}}} \mathcal{L}_\ell(x^{(\ell)}, y^{(\ell)}, c^{(\ell)}; \xi^{(\ell)}) \\ &= \sum_{b \in \mathcal{B}} \min_{c_b^{(\ell)} \geq 0} \left\{ T_3 c_b^{(\ell)} \log_2 c_b^{(\ell)} - \xi_b^{(\ell)} c_b^{(\ell)} \right\} \\ &\quad + \sum_{i \in \mathcal{I}_\ell} \min_{(x_i^{(\ell)}, y_i^{(\ell)}) \in \mathcal{F}_i^{(\ell)}} \left\{ T_1 \frac{(\bar{w}_i^{(\ell)})^2}{y_i^{(\ell)}} + T_2 y_i^{(\ell)} + \sum_{b \in \mathcal{B}} \xi_b^{(\ell)} x_{i,b}^{(\ell)} \right\}, \end{aligned} \quad (17)$$

C.3 SOLVING THE PER-WEIGHT SUBPROBLEM

Fix a tensor $\ell \in \Lambda$. Throughout this section, we temporarily omit the superscript (ℓ) , with the understanding that all quantities refer to the same tensor. Accordingly, $\xi = (\xi_b)_{b \in \mathcal{B}}$ denotes the multiplier vector introduced in Section 3.1 with its tensor superscript omitted.

For fixed ξ , the problem separates over the weight index i . Given a weight $\bar{w}_i$ and a fixed mass $y_i \in [0, 1]$, define

$$K_i(y_i; \bar{w}_i, \xi, v) := \min_{x_i} \sum_{b \in \mathcal{B}} \xi_b x_{i,b} \quad (18)$$

$$\text{s.t.} \quad \begin{cases} \sum_{b \in \mathcal{B}} v_b x_{i,b} = \bar{w}_i, & \sum_{b \in \mathcal{B}} x_{i,b} = y_i, \\ x_{i,b} \geq 0, & b \in \mathcal{B}. \end{cases} \quad (19)$$

The letter K denotes the optimal value of this linear knapsack-type subproblem, and the subscript i identifies the corresponding weight. Here, $x_i = (x_{i,b})_{b \in \mathcal{B}}$ is the vector of assignment masses for weight i , and $v = (v_b)_{b \in \mathcal{B}}$ is the vector of real-valued levels.

For $y_i > 0$, let

$$u_{i,b} := \frac{x_{i,b}}{y_i}, \quad \tilde{w}_i := \frac{\bar{w}_i}{y_i}.$$

The vector $u_i = (u_{i,b})_{b \in \mathcal{B}}$ has nonnegative entries summing to one, while $\tilde{w}_i$ is the rescaled target. It follows that

$$K_i(y_i; \bar{w}_i, \xi, v) = y_i K_i(1; \tilde{w}_i, \xi, v). \quad (20)$$

To evaluate $K_i(1; \tilde{w}_i, \xi, v)$, we associate each symbol b with the point $P_b := (v_b, \xi_b) \in \mathbb{R}^2$. The optimal value is the ordinate of the lower convex hull of the points P_b at the abscissa $v = \tilde{w}_i$.

Let E_l and E_r denote, respectively, the leftmost and rightmost vertices of the lower hull, as shown in Figure 3. Suppose first that $v_{b_-} < \tilde{w}_i < v_{b_+}$, where P_{b_-} and P_{b_+} are the endpoints of the lower-hull facet containing this abscissa. We call these points active because they determine the facet and are the only symbols assigned positive mass in the corresponding two-point optimal representation. If $\tilde{w}_i$ coincides with a lower-hull vertex v_b , the optimal representation assigns unit mass to symbol b . If P_b is an interior vertex of the lower hull, either of its two incident facets may be used in the formulas below. If P_b is one of the two extreme vertices E_l and E_r , only one incident facet exists. In both cases, the formulas yield unit mass for symbol b and zero mass for the other vertex of the selected facet.

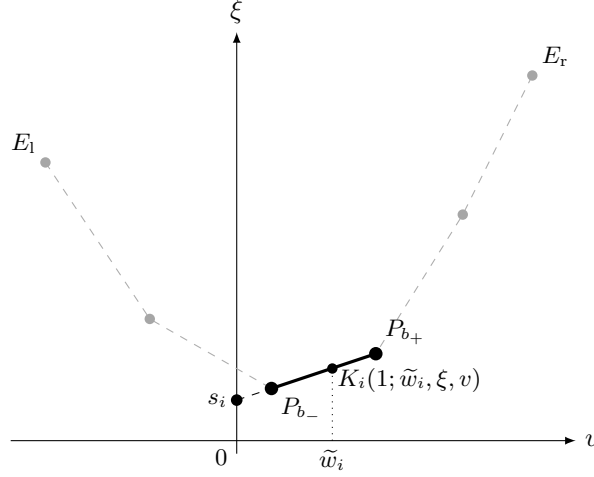


Figure 3: Geometric solution of the normalized per-weight subproblem.

Consider the two-point optimal representation associated with the selected facet. All masses outside its endpoints are set to zero, while the endpoint masses satisfy

$$u_{i,b_-} + u_{i,b_+} = 1, \quad v_{b_-} u_{i,b_-} + v_{b_+} u_{i,b_+} = \tilde{w}_i.$$

Solving this two-equation linear system yields

$$u_{i,b_-} = \frac{v_{b_+} - \tilde{w}_i}{v_{b_+} - v_{b_-}}, \quad u_{i,b_+} = \frac{\tilde{w}_i - v_{b_-}}{v_{b_+} - v_{b_-}}. \quad (21)$$

Both masses are strictly positive when $\tilde{w}_i$ lies in the interior of the facet; if it coincides with an endpoint, the corresponding masses are 1 and 0.

The line containing the active facet can be written as $\xi = \mu_i v + s_i$. Requiring this line to pass through P_{b_-} and P_{b_+} yields

$$\mu_i := \frac{\xi_{b_+} - \xi_{b_-}}{v_{b_+} - v_{b_-}}, \quad s_i := \frac{\xi_{b_-} v_{b_+} - \xi_{b_+} v_{b_-}}{v_{b_+} - v_{b_-}}. \quad (22)$$

Therefore,

$$K_i(1; \tilde{w}_i, \xi, v) = \mu_i \tilde{w}_i + s_i. \quad (23)$$

Breakpoints of the active facet. The facet that supports $\tilde{w}_i = \bar{w}_i/y_i$ changes only when $\tilde{w}_i$ crosses a vertex of the lower hull. Let $v_{(1)}, \dots, v_{(m)}$ be the abscissas of the hull vertices that have the same sign as $\bar{w}_i$, sorted by increasing magnitude. As y_i decreases from 1 to $y_i^{\min}$, the ratio $|\tilde{w}_i| = |\bar{w}_i|/y_i$ increases monotonically from $|\bar{w}_i|$ to $V_i^{\max}$. The active facet therefore changes exactly at

$$y_{(k)} := \frac{|\bar{w}_i|}{|v_{(k)}|}, \quad y_{(k)} \in [y_i^{\min}, 1], \quad (24)$$

and these values, together with $y_i^{\min}$ and 1, partition the feasible interval into at most $m \leq C$ subintervals. The pair (μ_i, s_i) is constant on each subinterval, so Eq. (26) applies there unchanged. The breakpoints require no search: they are obtained in closed form from the hull vertices, which are shared by every weight of the tensor.

Cost of the comparison. The lower hull of the C points (v_b, ξ_b) does not depend on i . It is built once per tensor and per dual iteration, in $O(C)$ time by a monotone chain, because the codebook values are already sorted. For a single weight, the initial facet is located by binary search in $O(\log C)$; the sweep then visits at most $m \leq C$ subintervals and performs constant work in each, namely one evaluation of Eq. (26), one clipping test, and one evaluation of G_i . The per-weight cost is therefore $O(C)$ in the worst case and $O(\log C)$ when the feasible interval spans few facets. One pass over a tensor with n_ℓ weights costs $O(C + n_\ell C)$, and a dual update with N_{dual} iterations costs N_{dual} such passes. Since C is a small constant (16 in all our experiments) and the sweep is vectorized over weights, the measured cost of the solver is a small multiple of one elementwise pass over the tensor; Section 4 reports the wall-clock effect.

To characterize the feasible interval of y_i , define

$$V_i^{\max} := \max\{|v_b| : b \in \mathcal{B}, \bar{w}_i v_b \geq 0\}.$$

If $\bar{w}_i = 0$, define $y_i^{\min} := 0$; otherwise, define

$$y_i^{\min} := \frac{|\bar{w}_i|}{V_i^{\max}}.$$

$V_i^{\max}$ is the largest representable magnitude having the same sign as $\bar{w}_i$, whereas $y_i^{\min}$ is the smallest mass that can represent $\bar{w}_i$.

It remains to minimize the scalar function

$$G_i(y_i) := K_i(y_i; \bar{w}_i, \xi, v) + T_1 \frac{\bar{w}_i^2}{y_i} + T_2 y_i, \quad y_i \in [y_i^{\min}, 1], \quad (25)$$

where G_i is the value of the subproblem associated with weight i . On an interval over which the active facet remains unchanged, $K_i(y_i; \bar{w}_i, \xi, v) = \mu_i \bar{w}_i + s_i y_i$. If $s_i + T_2 > 0$, the unique stationary candidate in the interior of that interval is

$$y_i^{\text{cand}} := |\bar{w}_i| \sqrt{\frac{T_1}{s_i + T_2}}. \quad (26)$$

Here, s_i is not an optimized quantity. For each candidate interval, it is computed from the lower-hull facet that remains active throughout that interval, using Eq. (22), before y_i^{cand} is evaluated. The candidates obtained from all active facets are compared only afterward to select y_i^* . The algorithm compares every candidate y_i^{cand} lying within its corresponding interval against the interval endpoints and the endpoints of $[y_i^{\min}, 1]$. We denote the resulting minimizer by y_i^* . The optimal mass assigned to the zero symbol is $z_i^* := 1 - y_i^*$.

The zero weight. One case has to be singled out. If $\bar{w}_i = 0$ then $y_i^{\min} = 0$, the whole feasible interval is available and, whenever the active prices are non-negative, the minimizer is $y_i^* = 0$ with $x_i = 0$, so the expression $\mu_i + 2T_1 \bar{w}_i / y_i^*$ of Eq. (12) is $0/0$ and does not define a value. It is a genuine kink rather than an artefact: along the stationary branch $y_i = |\bar{w}_i| \sqrt{T_1 / (s_i + T_2)}$ the perspective part tends to $\pm 2\sqrt{T_1(s_i + T_2)}$ as $\bar{w}_i \rightarrow 0^\pm$, so the two one-sided limits bracket the origin and the subdifferential of G_i^* at $\bar{w}_i = 0$ is an interval containing zero. The implementation selects $\beta_i^* = 0$ there, the midpoint of that interval, and Algorithm 1 states the branch explicitly.

C.4 REGULARIZER SUBGRADIENT WITH RESPECT TO THE WEIGHTS

Restoring the tensor superscript ℓ , let $\mu_i^{(\ell)}$ be an optimal Lagrange multiplier associated with the representation constraint of Eq. (4) for weight i , under the sign convention

$$\bar{w}_i^{(\ell)} - \sum_{b \in \mathcal{B}} a_\ell q_b x_{i,b}^{(\ell)} = 0.$$

The full inner Lagrangian, obtained by adding the representation constraints to the partial Lagrangian in Eq. (16), is

$$\mathcal{L}_\ell^{\text{full}} := \mathcal{L}_\ell + \sum_{i \in \mathcal{I}_\ell} \mu_i^{(\ell)} \left(\bar{w}_i^{(\ell)} - \sum_{b \in \mathcal{B}} a_\ell q_b x_{i,b}^{(\ell)} \right).$$

For a convex parametric optimization problem the optimal Lagrange multipliers describe how the optimal-value function responds to perturbations of the constraints, and when the parameter also appears explicitly in the objective its direct contribution has to be added to the contribution of the multiplier (Bonnans and Shapiro, 2000).

The connection with the per-weight function G_i in Eq. (25) can be seen directly. Temporarily omitting the tensor superscript, Eqs. (20) and (23) give, on the selected facet,

$$K_i(y_i; \bar{w}_i, \xi, v) = \mu_i \bar{w}_i + s_i y_i.$$

Consequently,

$$\mu_i \in \partial_{\bar{w}_i} K_i(y_i; \bar{w}_i, \xi, v),$$

where the inclusion accounts for a possible lower-hull breakpoint. From Eq. (25) we obtain, for fixed y_i ,

$$\mu_i + 2T_1 \frac{\bar{w}_i}{y_i} \in \partial_{\bar{w}_i} G_i(y_i).$$

To make the minimization over y_i explicit, define the optimized per-weight value

$$G_i^*(\bar{w}_i; \xi, v) := \min_{y_i \in [y_i^{\min}, 1]} G_i(y_i; \bar{w}_i, \xi, v), \quad y_i^* \in \arg \min_{y_i \in [y_i^{\min}, 1]} G_i(y_i; \bar{w}_i, \xi, v),$$

In this notation G_i^* denotes the optimal value while y_i^* denotes a selected minimizer. The dependence of G_i on $\bar{w}_i$, ξ , and v was suppressed in Eq. (25). At a differentiability point where y_i^* is an interior minimizer, formally applying the chain rule to $G_i^*(\bar{w}_i; \xi, v) = G_i(y_i^*(\bar{w}_i); \bar{w}_i, \xi, v)$ gives

$$\frac{dG_i^*}{d\bar{w}_i} = \frac{\partial G_i}{\partial \bar{w}_i} + \frac{\partial G_i}{\partial y_i} \frac{dy_i^*}{d\bar{w}_i} = \frac{\partial G_i}{\partial \bar{w}_i},$$

because $\frac{\partial G_i}{\partial y_i} \Big|_{y_i=y_i^*} = 0$ at the interior optimum. Danskin’s theorem and the sensitivity theory of convex value functions extend this conclusion to lower-hull breakpoints and boundary minimizers: a compatible subgradient is obtained from a selected optimal primal–dual solution while holding y_i^* fixed (Danskin, 1966; Bonnans and Shapiro, 2000). Therefore, no derivative of y_i^* is required, and

$$\mu_i + 2T_1 \frac{\bar{w}_i}{y_i^*} \in \partial_{\bar{w}_i} G_i^*(\bar{w}_i; \xi, v).$$

Restoring the tensor superscript, and observing that the count subproblems contain no explicit dependence on $\bar{w}_i^{(\ell)}$, yields

$$\beta_i^{(\ell)*} := \mu_i^{(\ell)} + 2T_1 \frac{\bar{w}_i^{(\ell)}}{y_i^{(\ell)*}} \in \partial_{\bar{w}_i^{(\ell)}} \Phi_\ell(\bar{w}^{(\ell)}, a_\ell) \Big|_{a_\ell}. \quad (27)$$

Thus, $\mu_i^{(\ell)}$ is the contribution induced by the representation constraint, whereas the second term is the direct contribution of the perspective penalty. Everything in this subsection and the next is written at a dual optimum, where $\varphi^{(\ell)}$ and Φ_ℓ coincide; the implementation evaluates the same expressions at the multiplier it currently holds, and Appendix C.6 states what they are exact for in that case. If Φ_ℓ is differentiable with respect to $\bar{w}_i^{(\ell)}$, the subgradient is unique and

$$\beta_i^{(\ell)*} = \frac{\partial \Phi_\ell}{\partial \bar{w}_i^{(\ell)}} \Big|_{a_\ell}.$$

At points where the active facet changes, Φ_ℓ may be nondifferentiable; the preceding quantities should then be interpreted as subgradients associated with either adjacent active facet. If the optimal normalized value reaches the extreme level $v_e^{(\ell)}$, namely $\bar{w}_i^{(\ell)}/y_i^{(\ell)*} = v_e^{(\ell)}$, representability forces all nonzero mass onto the corresponding extreme symbol $e \in \mathcal{B}$. The KKT conditions then yield

$$\mu_i^{(\ell)} = \frac{\xi_e^{(\ell)} + T_2}{v_e^{(\ell)}} - T_1 v_e^{(\ell)}. \quad (28)$$

when $y_i^{(\ell)*} = y_i^{(\ell), \min} < 1$. If the weight itself coincides with the extreme level, so that $y_i^{(\ell)*} = 1$, Eq. (28) selects the limiting multiplier obtained by approaching the endpoint from the interior of the representable interval. Here, $\xi_e^{(\ell)}$ is the dual multiplier associated with the count-defining constraint for symbol e in Eq. (6). Appendix C.6 derives Eq. (28) and explains the endpoint selection.

C.5 REGULARIZER GRADIENT WITH RESPECT TO THE STEP SIZES

Holding $\bar{w}^{(\ell)}$ fixed, the step size a_ℓ appears explicitly in the full inner Lagrangian $\mathcal{L}_\ell^{\text{full}}$ only through the representation-constraint term

$$-\sum_{i \in \mathcal{I}_\ell} \mu_i^{(\ell)} \sum_{b \in \mathcal{B}} a_\ell q_b x_{i,b}^{(\ell)},$$

because $v_b^{(\ell)}(a_\ell) = a_\ell q_b$. The envelope theorem therefore computes the derivative of the optimal-value function by differentiating this explicit dependence at an optimal primal-dual solution; derivatives of the optimal assignments $x_{i,b}^{(\ell)*}$ with respect to a_ℓ are not required (Bonnans and Shapiro, 2000). Since $\partial v_b^{(\ell)} / \partial a_\ell = q_b$, this gives

$$\left. \frac{\partial \Phi_\ell}{\partial a_\ell} \right|_{\bar{w}^{(\ell)}} = -\sum_{i \in \mathcal{I}_\ell} \mu_i^{(\ell)} \sum_{b \in \mathcal{B}} q_b x_{i,b}^{(\ell)*} = -\frac{1}{a_\ell} \sum_{i \in \mathcal{I}_\ell} \mu_i^{(\ell)} \bar{w}_i^{(\ell)}. \quad (29)$$

Thus, the first equality follows from the envelope theorem. The second follows from the representation constraint in Eq. (4), together with $v_b^{(\ell)} = a_\ell q_b$.

C.6 KKT MULTIPLIER AT A REPRESENTABILITY BOUNDARY

Here we derive the representation multiplier reported in Eq. (28). Fix a tensor ℓ and a weight $i \in \mathcal{I}_\ell$, and suppose that its optimal mass attains the representability lower bound

$$y_i^{(\ell)*} = y_i^{(\ell),\min} = \frac{|\bar{w}_i^{(\ell)}|}{|v_e^{(\ell)}|},$$

where $v_e^{(\ell)}$ is the extreme codebook level having the same sign as $\bar{w}_i^{(\ell)}$. Equivalently,

$$\frac{\bar{w}_i^{(\ell)}}{y_i^{(\ell)*}} = v_e^{(\ell)}.$$

Because no codebook level extends beyond $v_e^{(\ell)}$ in that direction, the representation and mass constraints can hold only if all nonzero mass is assigned to symbol e :

$$x_{i,e}^{(\ell)*} = y_i^{(\ell)*}, \quad x_{i,b}^{(\ell)*} = 0 \quad \text{for every } b \neq e.$$

Let $\rho_i^{(\ell)} \in \mathbb{R}$ be the Lagrange multiplier associated with the mass constraint

$$\sum_{b \in \mathcal{B}} x_{i,b}^{(\ell)} - y_i^{(\ell)} = 0.$$

For every $b \in \mathcal{B}$, let $\gamma_{i,b}^{(\ell)} \geq 0$ be the multiplier associated with the nonnegativity constraint written as $-x_{i,b}^{(\ell)} \leq 0$. Using for the representation constraint the sign convention introduced in Appendix C.4, define the per-weight KKT Lagrangian relevant to the derivation as

$$\begin{aligned} \mathcal{L}_{i,\ell}^{\text{KKT}} := & T_1 \frac{(\bar{w}_i^{(\ell)})^2}{y_i^{(\ell)}} + T_2 y_i^{(\ell)} + \sum_{b \in \mathcal{B}} \xi_b^{(\ell)} x_{i,b}^{(\ell)} \\ & + \mu_i^{(\ell)} \left(\bar{w}_i^{(\ell)} - \sum_{b \in \mathcal{B}} v_b^{(\ell)} x_{i,b}^{(\ell)} \right) + \rho_i^{(\ell)} \left(\sum_{b \in \mathcal{B}} x_{i,b}^{(\ell)} - y_i^{(\ell)} \right) - \sum_{b \in \mathcal{B}} \gamma_{i,b}^{(\ell)} x_{i,b}^{(\ell)}. \end{aligned}$$

Complementary slackness requires $\gamma_{i,e}^{(\ell)} x_{i,e}^{(\ell)*} = 0$; since $x_{i,e}^{(\ell)*} > 0$, it follows that $\gamma_{i,e}^{(\ell)} = 0$.² Stationarity with respect to the active mass, evaluated at the selected optimum, therefore gives

$$\left. \frac{\partial \mathcal{L}_{i,\ell}^{\text{KKT}}}{\partial x_{i,e}^{(\ell)}} \right|_{\substack{x_{i,e}^{(\ell)} = x_{i,e}^{(\ell)*} \\ y_i^{(\ell)} = y_i^{(\ell)*}}} = \xi_e^{(\ell)} - \mu_i^{(\ell)} v_e^{(\ell)} + \rho_i^{(\ell)} - \gamma_{i,e}^{(\ell)} = \xi_e^{(\ell)} - \mu_i^{(\ell)} v_e^{(\ell)} + \rho_i^{(\ell)} = 0. \quad (30)$$

²For an inequality written as $-x_{i,e}^{(\ell)} \leq 0$, the KKT complementary-slackness condition is $\gamma_{i,e}^{(\ell)} x_{i,e}^{(\ell)*} = 0$. Thus, strict positivity of the primal variable forces the associated nonnegativity multiplier to vanish; see Bonnans and Shapiro (2000).

When $y_i^{(\ell)*} < 1$, the upper bound $y_i^{(\ell)} \leq 1$ is inactive. Stationarity with respect to $y_i^{(\ell)}$ then yields

$$-T_1 \frac{(\bar{w}_i^{(\ell)})^2}{(y_i^{(\ell)*})^2} + T_2 - \rho_i^{(\ell)} = 0. \quad (31)$$

Using $\bar{w}_i^{(\ell)} / y_i^{(\ell)*} = v_e^{(\ell)}$ in Eq. (31) gives

$$\rho_i^{(\ell)} = T_2 - T_1 (v_e^{(\ell)})^2.$$

Substitution into Eq. (30) produces

$$\mu_i^{(\ell)} = \frac{\xi_e^{(\ell)} + T_2}{v_e^{(\ell)}} - T_1 v_e^{(\ell)},$$

which is Eq. (28).

The endpoint case remains to be clarified. If $\bar{w}_i^{(\ell)} = v_e^{(\ell)}$, then representability forces $y_i^{(\ell)*} = 1$ and the upper-bound multiplier for $y_i^{(\ell)} \leq 1$ may be nonzero, so that the representation multiplier is not necessarily unique, and in that situation Equation (28) specifies the multiplier obtained by setting that upper-bound multiplier to zero, equivalently the limit of the preceding expression as $\bar{w}_i^{(\ell)}$ approaches $v_e^{(\ell)}$ from the interior of the representable interval. This is the subgradient selection used in the optimization procedure.

C.7 COUNT SOLUTIONS AND DUAL RESIDUALS

Fix a tensor $\ell \in \Lambda$ and a multiplier vector $\xi^{(\ell)}$. For each symbol $b \in \mathcal{B}$, the solutions of the per-weight subproblems determine the reconstructed count

$$\hat{c}_b^{(\ell)} := \sum_{i \in \mathcal{I}_\ell} x_{i,b}^{(\ell)*}. \quad (32)$$

The hat distinguishes this quantity, which is reconstructed by summing the optimal assignment masses, from the independent variable $c_b^{(\ell)}$ in the count subproblem.

For $T_3 > 0$, the count subproblem appearing in the first line of Eq. (9) is

$$\min_{c_b^{(\ell)} \geq 0} \left\{ T_3 c_b^{(\ell)} \log_2 c_b^{(\ell)} - \xi_b^{(\ell)} c_b^{(\ell)} \right\}.$$

Its objective is strictly convex for $c_b^{(\ell)} > 0$ and its derivative on this interval is

$$\frac{T_3}{\ln(2)} (\ln c_b^{(\ell)} + 1) - \xi_b^{(\ell)}.$$

Setting this derivative to zero and solving for $c_b^{(\ell)}$ gives the unconstrained stationary value

$$c_{b,\text{unc}}^{(\ell)} = \exp \left(\frac{\ln(2) \xi_b^{(\ell)}}{T_3} - 1 \right). \quad (33)$$

For numerical stability, the implementation restricts the count to $[c_{\min}, n_\ell]$, where $c_{\min} > 0$ is a fixed numerical lower bound and n_ℓ is the number of weights in tensor ℓ . The value actually used by the dual solver is therefore

$$c_b^{(\ell)*} := \text{clip} \left(c_{b,\text{unc}}^{(\ell)}; c_{\min}, n_\ell \right). \quad (34)$$

When $T_3 = 0$, the entropic count component is disabled and Eqs. (33) and (34) are not used.

The two count values in Eqs. (32) and (34) solve different subproblems and need not agree for an arbitrary multiplier vector. Their difference

$$s_b^{(\ell)} := \hat{c}_b^{(\ell)} - c_b^{(\ell)*} \quad (35)$$

is precisely the residual of the relaxed count constraint. It is also a supergradient of the concave dual function with respect to $\xi_b^{(\ell)}$, and of the *boxed* dual $\varphi_\ell^\square$ rather than of $\varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}$, because the count entering it is the clipped $c_b^{(\ell)*}$ of Eq. (34) and not the unconstrained stationary value³. Thus, $s_b^{(\ell)} = 0$ means that the relaxed constraint is satisfied for symbol b . If a subproblem has multiple minimizers, any consistent selection yields a valid supergradient.

C.8 INNER DUAL ITERATION

Let N_{dual} be the positive integer specifying the number of dual iterations performed per solver call, let $t \in \{0, \dots, N_{\text{dual}} - 1\}$ index those iterations, and let $\eta_\xi > 0$ be the dual step size. At the multiplier vector $\xi^{(\ell, t)}$, the implementation first solves the per-weight and count subproblems and forms the normalized supergradient

$$g_b^{(\ell, t)} := \frac{\tilde{c}_b^{(\ell, t)} - c_b^{(\ell, t)*}}{n_\ell}, \quad b \in \mathcal{B}. \quad (36)$$

If the per-weight subproblems assign more mass to symbol b than the count subproblem does, then $g_b^{(\ell, t)} > 0$ and the corresponding multiplier is pushed upwards by dual ascent. Division by $n_\ell = |\mathcal{I}_\ell|$ normalizes the residual by the number of weights in the tensor, preventing the scale of the update from growing merely because a tensor contains more weights.

The dual solver used in our experiments applies a FISTA-style momentum extrapolation (Beck and Teboulle, 2009); it is not an optional alternative to the update above. To state the implemented iteration precisely, let $\tau_0 = 1$ and initialize $\xi^{(\ell, -1)} = \xi^{(\ell, 0)}$. At iteration t , the momentum parameter and extrapolated multiplier vector are

$$\tau_{t+1} := \frac{1 + \sqrt{1 + 4\tau_t^2}}{2}, \quad (37)$$

$$\tilde{\xi}^{(\ell, t)} := \xi^{(\ell, t)} + \frac{\tau_t - 1}{\tau_{t+1}} \left(\xi^{(\ell, t)} - \xi^{(\ell, t-1)} \right). \quad (38)$$

The normalized supergradient in Eq. (36) is then applied to the extrapolated point⁴:

$$z_b^{(\ell, t+1)} := \tilde{\xi}_b^{(\ell, t)} + \eta_\xi g_b^{(\ell, t)}, \quad b \in \mathcal{B}. \quad (39)$$

The count bounds in Eq. (34), together with the unconstrained count solution of Appendix C.7, correspond to the multiplier bounds

$$\xi_{\min}^{(\ell)} := \frac{T_3}{\ln(2)} (\ln c_{\min} + 1), \quad \xi_{\max}^{(\ell)} := \frac{T_3}{\ln(2)} (\ln n_\ell + 1). \quad (40)$$

After Eq. (39), every component is clipped to this interval:

$$\xi^{(\ell, t+1)} := \text{clip}\left(z^{(\ell, t+1)}; \xi_{\min}^{(\ell)}, \xi_{\max}^{(\ell)}\right). \quad (41)$$

Here, clip acts componentwise, so each $\xi_b^{(\ell)}$ remains inside $[\xi_{\min}^{(\ell)}, \xi_{\max}^{(\ell)}]$. The per-weight subproblems, the count solution, the residual, the extrapolation and the final clipping are repeated for the prescribed number of dual iterations N_{dual} .

This is a FISTA-style rather than a literal canonical FISTA iteration: the supergradient $g^{(\ell, t)}$ is evaluated at $\xi^{(\ell, t)}$, whereas its correction is applied to the extrapolated vector $\tilde{\xi}^{(\ell, t)}$. Stating both evaluation points is necessary to reproduce the implemented update unambiguously.

Once the N_{dual} dual iterations are completed, the per-weight solutions x^* and y^* , together with the associated quantities μ and β^* , computed during the last iteration are used in the PRESTO-gradient formulas of Section 3.2.⁵

³The coefficient of $\xi_b^{(\ell)}$ in the partial Lagrangian is $\sum_i x_{i,b}^{(\ell)} - c_b^{(\ell)}$, and evaluating this coefficient at the minimizers selected under the box gives Eq. (35); the two duals coincide where the box is inactive.

⁴The plus sign follows from the fact that Eq. (11) is a maximization problem, so the update performs supergradient ascent.

⁵A finite N_{dual} does not, in general, produce the exact maximizer of the dual problem in Eq. (11). It is a computational budget that trades the quality of the approximate dual solution against the cost of each

C.9 DUAL STEP SIZE AND CONVERGENCE RATE

The entropy coefficient reaches the latent weights through one channel only. The per-weight subproblem is solved against the raw multipliers $\xi_b^{(\ell)}$, and T_3 does not appear in it: it enters the dual decomposition only through the count subproblem. The entropy term is therefore exactly as strong as the dual is converged, and no stronger.

This creates a scale coupling that is easy to miss. The residual of Eq. (36) is normalized to $[-1, 1]$, so an absolute step η moves the multiplier by a fixed distance per iteration. The interval in which the multiplier is useful, however, has width $T_3 \log_2(n_\ell/c_{\min})$, which is proportional to T_3 . Raising the coefficient therefore moves the target away at the same rate at which it raises it, and we measured that effect directly.

Let one *traversal* denote the number of times the multiplier could cross its own interval in one epoch, that is $\eta \bar{g} N_{\text{dual}} S / (T_3 \log_2(n_\ell/c_{\min}))$, where S is the number of dual calls per epoch and $\bar{g} \approx 1/C$ is the typical residual. Table 11 reports it for the runs of Section 4.

Table 11: Traversals of the dual interval per epoch. The first four rows are computed at a fixed absolute dual step of 3×10^{-9} , the setting under which the failure was found; they are the diagnostic that motivated the relative step and not the setting of the runs reported in Section 4, every one of which uses the relative step, as the last row does.

Network	T_3	Dual calls / epoch	Traversals / epoch	Outcome
ResNet-18	3×10^{-8}	313	0.195	entropy term effective
DeiT-Small	1.5×10^{-8}	156	0.189	entropy term effective
EfficientNet-B0	3×10^{-7}	78	0.0050	inert
EfficientNet-B0	3×10^{-6}	78	0.0005	inert
EfficientNet-B0	3×10^{-8}	78	0.76	effective

The two inert rows differ tenfold in T_3 , and the norms of the applied regularizer gradient at the first epoch agree to four significant figures, 2.756366×10^{-4} against 2.756720×10^{-4} . Both compressed less than the same recipe with the entropy term switched off. The last row uses the same coefficient as the first row of the table and only changes the step to a fraction of the interval; it recovers a working entropy term, with four times the control’s reduction in packet size, symbol entropy and sparsity, at 0.14 accuracy points.

The practical rule is that η should be expressed as a fraction of the dual interval rather than as an absolute distance. It then follows T_3 , the tensor size, and the number of dual calls per epoch without further tuning.

C.10 STRONG DUALITY OF THE RELAXATION

The relaxation of Section 3.1 is exact, and the condition under which it is exact is weaker than Slater’s because every constraint of the inner problem is affine.

Proposition 1 (No duality gap, attained multipliers). *Fix a tensor ℓ , a step size $a_\ell > 0$ and a clipped weight vector $\bar{w}^{(\ell)}$ satisfying $|\bar{w}_i^{(\ell)}| \leq V_i^{\max}$ for every $i \in \mathcal{I}_\ell$, with the inequality strict for at least one i . Then the inner problem of Eq. (3) has a finite optimal value, its minimum is attained, and the partial dual obtained by relaxing only the count constraints (6) has no gap:*

$$\Phi_\ell(\bar{w}^{(\ell)}, a_\ell) = \max_{\xi^{(\ell)} \in \mathbb{R}^B} \varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}(\xi^{(\ell)}),$$

with the maximum attained on a nonempty set of optimal multipliers.

Proof. Convexity, feasibility and attainment. With the closed extension adopted in Section 2.2, $(u, y) \mapsto u^2/y$ is the perspective of $t \mapsto t^2$ and is therefore jointly convex and closed on $\mathbb{R} \times \mathbb{R}_{\geq 0}$;

training update, and is therefore an important hyperparameter. To mitigate the use of a small budget, each dual-solver call is warm-started from the multiplier vector returned by the preceding call rather than from a new initialization. This allows progressive refinement across calls, although the underlying weights and codebook may change between them. The dependence on N_{dual} and the effect of warm starting are analyzed in Appendix C.9.

$c \mapsto c \log_2 c$ is convex and closed on $\mathbb{R}_{\geq 0}$ with the convention $0 \log_2 0 = 0$; the T_2 term is linear. The objective f is therefore closed, proper and convex, the constraints (4), (5) and (6) are affine and the domain (7) is a box, so the feasible set is a polyhedron, and it is bounded because $0 \leq x_{i,b}^{(\ell)} \leq y_i^{(\ell)} \leq 1$ and $c^{(\ell)}$ is determined by $x^{(\ell)}$. Clipping supplies a feasible point of finite value: for $\bar{w}_i^{(\ell)} \neq 0$ place all of the mass of weight i on the extreme level $b_i^{\max}$ attaining $V_i^{\max}$, so that $y_i^{(\ell)} = |\bar{w}_i^{(\ell)}|/V_i^{\max} \leq 1$ by hypothesis, and take $y_i^{(\ell)} = 0$ where $\bar{w}_i^{(\ell)} = 0$. The primal value is therefore finite and, the feasible set being compact and f closed, the minimum is attained.

No duality gap. Let

$$v(u) := \min \left\{ f : (x^{(\ell)}, y^{(\ell)}) \in \prod_i \mathcal{F}_i^{(\ell)}, c^{(\ell)} \geq 0, \sum_i x_{i,b}^{(\ell)} - c_b^{(\ell)} = u_b \forall b \right\}$$

be the value of the problem in which the relaxed constraints are perturbed by $u \in \mathbb{R}^B$. The function v is convex and $v(0)$ is finite by the previous paragraph, and it is lower semicontinuous at the origin: if $u^k \rightarrow 0$ and z^k attains $v(u^k)$, the points z^k lie in a fixed compact set, so a subsequence converges to a z that satisfies the constraints at $u = 0$, and lower semicontinuity of f gives $v(0) \leq f(z) \leq \liminf_k v(u^k)$. A convex function that is finite and lower semicontinuous at a point agrees there with its biconjugate (Rockafellar, 1970), and the optimal value of the partial dual is exactly $v^{**}(0)$; hence the two values coincide.

Attainment of the multiplier. The set of dual optima is $-\partial v(0)$, which is nonempty and compact whenever $0 \in \text{int dom } v$. Let i be a weight whose clipped value lies strictly inside the representable interval, which exists by hypothesis. Then $\bar{w}_i^{(\ell)}$ is a convex combination of the levels with *all* coefficients strictly positive, so spreading the mass of that single weight accordingly gives a feasible point with $\sum_i x_{i,b}^{(\ell)} > 0$ for every $b \in \mathcal{B}$. Every sufficiently small perturbation u is then feasible, so the origin is interior to $\text{dom } v$ and the dual optimum is attained on a nonempty bounded set. $\square$

Two consequences of the proposition are used elsewhere in the paper. Because the minimum in the definition of $\varphi^{(\ell)}$ is attained over a compact set, Danskin’s theorem (Danskin, 1966) applies and the derivatives of Appendices C.4 and C.5 are envelope derivatives at the inner optimum rather than derivatives of a smoothed surrogate. Where the inner minimizer is unique, which holds in y whenever $T_1 > 0$ and $\bar{w}_i^{(\ell)} \neq 0$ by strict convexity of the perspective term, those expressions are gradients, whereas at a lower-hull breakpoint or at the representability boundary the minimizer may fail to be unique and they are then the particular subgradient selected above.

C.11 WHAT THE ENTROPY TERM DOES

Write $m = \sum_{b \in \mathcal{B}} c_b^{(\ell)}$ for the non-zero mass and $p_b = c_b^{(\ell)}/m$ for the conditional distribution over $\mathcal{B}$. The ideal zero-order code length of the tensor decomposes as

$$n_\ell H(m/n_\ell) + m H(p), \quad (42)$$

the support mask plus the value stream, which is n_ℓ times the entropy of the distribution over all C_ℓ symbols, the zero symbol included. This is a proxy and not the size the coder produces: a real coder carries container overhead and can exploit structure that a symbol histogram does not describe, which is why the mask costs 2.905% where the proxy predicts 3.12% (Appendix A.4). The value stream goes the same way on AlexNet, where the zero-order entropy of the non-zero symbols is 1.686 bits while the coder delivers those symbols at 1.34, the difference being structure that a symbol histogram does not describe. Every size we report is a measurement; the proxy is what the objective can be written against. The entropy term acts on the first of those two channels. The scalar map $c \mapsto c \log_2 c$ is minimized at $c = 1/e$, so the unconstrained minimizer of $\sum_b c_b^{(\ell)} \log_2 c_b^{(\ell)}$ assigns a count of $1/e$ to every non-zero symbol, an $O(1)$ occupancy that is negligible against a tensor of n_ℓ weights: the penalty prices membership of $\mathcal{B}$ and drives mass into the excluded zero symbol, which lowers $H(m/n_\ell)$ and therefore the entropy of the full symbol distribution. At a fixed m it acts in the opposite direction, because the same sum is convex and symmetric and is therefore minimized at uniform counts, and the identity $mH(p) = m \log_2 m - \sum_b c_b^{(\ell)} \log_2 c_b^{(\ell)}$ makes that explicit: at fixed mass, minimizing our term maximizes $mH(p)$. The two effects are of different size, $m \log_2 m$ against $m \log_2 B_\ell$ with $B_\ell = 15$ and m in the millions, so the mass channel is the larger by a factor

of five to six, which is the direction the term ablation measures. We state this because the reduction we report in the entropy of the non-zero symbols is an empirical consequence of the sparsity the term creates and not a property of the objective; a term that directly minimized $mH(p)$ would be concave in the counts, and neither the decomposition of Section 3.1 nor Proposition 1 would survive it.

C.12 THE PROBLEM THE SOLVER POSES, AND WHAT THE DIRECTION IS EXACT FOR

The inner problem of Eq. (3) lets $c_b^{(\ell)}$ range over $[0, \infty)$, whereas the implementation confines it to $[c_{\min}, n_\ell]$ with $c_{\min} = 10^{-2}$ and projects the multiplier onto the matching interval of Eq. (40). That box is not cosmetic and we do not treat it as such. Write $\Phi_\ell^\square$ for the inner problem of Eq. (3) with $c^{(\ell)} \in [c_{\min}, n_\ell]^B$ imposed, and $\varphi_\ell^\square(\xi^{(\ell)})$ for the partial dual obtained from it by relaxing the count constraints exactly as before. Restricting the feasible set can only raise the minimum, so $\Phi_\ell^\square \geq \Phi_\ell$, and what the solver works with is $\varphi_\ell^\square$, not $\varphi^{(\ell)}$; the two problems coincide when the box is inactive. The multiplier is boxed as well. Writing

$$\Xi_\ell := [\xi_{\min}^{(\ell)}, \xi_{\max}^{(\ell)}]^B$$

for the interval of Eq. (40), onto which Eq. (41) projects after every step, the iteration is an ascent on $\varphi_\ell^\square$ restricted to Ξ_ℓ , so the value it can reach is $\max_{\xi \in \Xi_\ell} \varphi_\ell^\square(\xi)$, which by weak duality satisfies

$$\max_{\xi \in \Xi_\ell} \varphi_\ell^\square(\xi) \leq \max_{\xi \in \mathbb{R}^B} \varphi_\ell^\square(\xi) = \Phi_\ell^\square,$$

with equality when the projection is inactive at a dual optimum. Since $\xi_{\min}^{(\ell)}$ and $\xi_{\max}^{(\ell)}$ are exactly the multipliers at which the unconstrained count solution meets $c_{\min}$ and n_ℓ , the projection is the dual face of the count box rather than a second, independent restriction; but it can bind, and we report below how often it does.

Proposition 1 carries over to $\Phi_\ell^\square = \max_{\xi^{(\ell)} \in \mathbb{R}^B} \varphi_\ell^\square(\xi^{(\ell)})$, the unprojected statement, but its hypothesis has to be strengthened, because the box shrinks the feasible set and feasibility must be re-established. Strict interiority alone does not suffice: a weight arbitrarily close to an endpoint of the representable interval does admit a strictly positive representation, but with coefficients that vanish as it approaches that endpoint, so it may be unable to carry $c_{\min}$ on every symbol while keeping $y_i^{(\ell)} \leq 1$. What suffices is a margin, and for a uniform codebook the margin can be written down. Give every symbol a floor η , which consumes mass $B\eta$ and represents the value $\eta a_\ell \sum_{b \in \mathcal{B}} q_b$; the mass left over, $1 - B\eta$, can then be placed on any single level, so a weight is representable with every coefficient at least η whenever

$$\eta \sum_{b \in \mathcal{B}} q_b + (1 - B\eta) Q_n \leq \frac{\bar{w}_i^{(\ell)}}{a_\ell} \leq \eta \sum_{b \in \mathcal{B}} q_b + (1 - B\eta) Q_p,$$

the endpoints being attained by putting all of the leftover mass on one extreme level and every interior value by a convex combination or by using less of it. With our grid, $Q_n = -8$, $Q_p = 7$, $B = 15$ and $\sum_b q_b = -8$, taking $\eta = 2c_{\min} = 2 \times 10^{-2}$ gives $\bar{w}_i^{(\ell)}/a_\ell \in [-5.76, 4.74]$, and one weight in that band places strictly more than $c_{\min}$ on each of the B symbols, which is what the perturbation argument of Proposition 1 needs. The existence of such a weight is the hypothesis of the boxed proposition. We state it as a hypothesis and not as an established property of our runs: it is checkable in one pass over a tensor, it concerns a single weight out of millions, and it fails only for a tensor whose entire weight vector sits beyond $\pm 5a_\ell$ on its own MSE-fitted grid. We did not instrument the runs to certify it, and nothing we compute depends on it, since the exactness statement below is made at the multiplier the solver holds.

The box is active, so the distinction is not a formality: the fraction of multipliers sitting on a bound at the end of an epoch reaches 0.003 on ResNet-18 at a hundred epochs, 0.09 on ViT-B/16, 0.31 on ResNet-50 and 0.56 on AlexNet. What we ascend is therefore the projected dual of the boxed problem, and every claim of exactness below is made against it.

What the training direction is exact for. The boxed proposition guarantees that

$$\Phi_\ell^\square(\bar{w}^{(\ell)}, a_\ell) = \max_{\xi^{(\ell)} \in \mathbb{R}^B} \varphi_\ell^\square(\xi^{(\ell)}) \geq \max_{\xi^{(\ell)} \in \Xi_\ell} \varphi_\ell^\square(\xi^{(\ell)}),$$

and the implementation attains neither maximum: it advances the multiplier inside Ξ_ℓ by N_{dual} warm-started supergradient steps per call, with $N_{\text{dual}} = 3$ in every run of this paper except the two ViT-B/16 frontier points, which use 6. Writing ξ for the multiplier the solver currently holds and ξ^* for a dual optimum, two statements have to be kept apart. The first is exact: at the fixed ξ the per-weight and count subproblems are solved to optimality, so the quantities of Appendices C.4 and C.5 belong to $\partial_{(\bar{w}^{(\ell)}, a_\ell)} \varphi_\ell^\square(\xi)$, the subdifferential of the boxed relaxed value at that multiplier, and $\varphi_\ell^\square(\xi)$ is the number the solver computes, not the maximized dual value. The second is not: since $\varphi_\ell^\square(\xi) \leq \max_{\Xi_\ell} \varphi_\ell^\square \leq \Phi_\ell^\square$ for every $\xi \in \Xi_\ell$ by weak duality, with equality throughout only at an unprojected dual optimum, those same quantities are in general *not* elements of $\partial \Phi_\ell^\square$, and the outer optimization is a descent on a Lagrangian lower bound of the boxed problem, driven by a direction that is an exact subgradient of that lower bound and only an inexact proxy for $\partial \Phi_\ell^\square$, to which it belongs when the dual is optimal. Neither statement is made against Φ_ℓ itself, from which $\Phi_\ell^\square$ differs by the box. The gap between $\varphi_\ell^\square(\xi)$ and $\Phi_\ell^\square$ is diagnosed by the dual residual, the difference between the counts reconstructed from the per-weight solutions and the counts the symbol subproblems return, which is the supergradient the dual update consumes. A zero residual certifies dual optimality, but the converse does not follow: the dual is concave and may be nondifferentiable, so optimality asks only that zero belong to the superdifferential and the residual produced by one selected primal minimizer need not vanish, and a multiplier held at a bound by the projection of Eq. (40) can be stationary for the projected dual with a non-zero residual, which is the second reason the residual is a diagnostic and not a certificate. We therefore use it as a diagnostic and do not claim an error bound.

Two properties of the scheme keep that inexactness from being a problem in practice, and one measurement shows what happens when they fail. The multiplier is warm-started across calls and the weights move slowly between calls, so the dual is not restarted at every step but tracks a slowly moving optimum, which is the standard setting of an incremental Lagrangian method rather than a fresh truncated solve. The dual step is expressed as a fraction of the dual range, so the number of times the multiplier can cross that range per epoch does not depend on T_3 , on the layer size or on the batch size, and Table 11 reports it for the runs of this paper. When that second property was absent the failure was visible and severe: with an absolute step, raising T_3 tenfold left the applied gradient identical to four significant figures and the model compressed *less*, which is the negative result reported in Appendix B.8. We therefore report the traversal rate rather than claim dual convergence.

C.13 TOTAL TRAINING GRADIENTS

Define the gradient of the task loss with respect to the quantized weight as

$$\delta_i^{(\ell)} := \frac{\partial \mathcal{L}_{\text{task}}}{\partial \hat{w}_i^{(\ell)}}.$$

$\delta_i^{(\ell)}$ is the upstream gradient returned by backpropagation. The LSQ straight-through estimator (STE) approximates⁶

$$\frac{\partial \hat{w}_i^{(\ell)}}{\partial w_i^{(\ell)}} \simeq \mathbf{1}\{Q_n \leq r_i^{(\ell)} \leq Q_p\}, \quad (43)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function, equal to one when its argument is true and zero otherwise, and $r_i^{(\ell)}$ is defined in Eq. (13). Under this estimator, the latent-weight component of the task-loss gradient is

$$\nabla_{w_i^{(\ell)}} \mathcal{L}_{\text{task}} \simeq \delta_i^{(\ell)} \mathbf{1}\{Q_n \leq r_i^{(\ell)} \leq Q_p\}. \quad (44)$$

We next derive the LSQ gradient with respect to the step size. If $r_i^{(\ell)} < Q_n$ or $r_i^{(\ell)} > Q_p$, clipping gives respectively $\hat{w}_i^{(\ell)} = a_\ell Q_n$ or $\hat{w}_i^{(\ell)} = a_\ell Q_p$; the corresponding derivatives with respect to a_ℓ are therefore Q_n and Q_p . Inside the representable interval,

$$\hat{w}_i^{(\ell)} = a_\ell \text{round}(r_i^{(\ell)}).$$

⁶The quantizer contains a nondifferentiable rounding operation. During QAT, the STE leaves the quantized forward pass unchanged but replaces the derivative of the quantizer in the backward pass with a surrogate derivative, thereby enabling gradient-based optimization; see Bengio et al. (2013); Esser et al. (2020).

Using the STE surrogate $\partial \text{round}(r)/\partial r \simeq 1$ gives

$$\frac{\partial \hat{w}_i^{(\ell)}}{\partial a_\ell} \simeq \text{round}(r_i^{(\ell)}) + a_\ell \frac{\partial r_i^{(\ell)}}{\partial a_\ell} = \text{round}(r_i^{(\ell)}) - r_i^{(\ell)}.$$

These three cases are collected in the function

$$h(r) := \begin{cases} Q_n, & r < Q_n, \\ \text{round}(r) - r, & Q_n \leq r \leq Q_p, \\ Q_p, & r > Q_p, \end{cases} \quad (45)$$

where $r \in \mathbb{R}$ is a scalar argument. Applying the chain rule and the definition of $\delta_i^{(\ell)}$ gives

$$\frac{\partial \mathcal{L}_{\text{task}}}{\partial a_\ell} \simeq \sum_{i \in \mathcal{I}_\ell} \frac{\partial \mathcal{L}_{\text{task}}}{\partial \hat{w}_i^{(\ell)}} \frac{\partial \hat{w}_i^{(\ell)}}{\partial a_\ell} = \sum_{i \in \mathcal{I}_\ell} \delta_i^{(\ell)} h(r_i^{(\ell)}).$$

Accordingly, the step-size component of the task-loss gradient is

$$\nabla_{a_\ell} \mathcal{L}_{\text{task}} \simeq \sum_{i \in \mathcal{I}_\ell} \delta_i^{(\ell)} h(r_i^{(\ell)}). \quad (46)$$

We must additionally account for the dependence of $\bar{w}_i^{(\ell)}(a_\ell)$ on the step size through the clipping operation in Eq. (14). Selected latent-weight and step-size components of the PRESTO subdifferential are

$$(\nabla_{\text{PRESTO}})_{w_i^{(\ell)}} := \beta_i^{(\ell)*} \mathbf{1}\{Q_n \leq r_i^{(\ell)} \leq Q_p\}, \quad (47)$$

$$(\nabla_{\text{PRESTO}})_{a_\ell} := -\frac{1}{a_\ell} \sum_{i \in \mathcal{I}_\ell} \mu_i^{(\ell)} \bar{w}_i^{(\ell)} + \sum_{\substack{i \in \mathcal{I}_\ell \\ r_i^{(\ell)} < Q_n}} \beta_i^{(\ell)*} Q_n + \sum_{\substack{i \in \mathcal{I}_\ell \\ r_i^{(\ell)} > Q_p}} \beta_i^{(\ell)*} Q_p. \quad (48)$$

The first term in Eq. (48) is the direct dependence of Φ_ℓ on a_ℓ at fixed $\bar{w}^{(\ell)}$, given by the step-size envelope derivative of Appendix C.5. There is also an indirect dependence through the clipped weights $\bar{w}_i^{(\ell)}(a_\ell)$. Holding the latent weight $w_i^{(\ell)}$ fixed, Eq. (14) gives

$$\frac{\partial \bar{w}_i^{(\ell)}}{\partial a_\ell} = \begin{cases} Q_n, & r_i^{(\ell)} < Q_n, \\ 0, & Q_n \leq r_i^{(\ell)} \leq Q_p, \\ Q_p, & r_i^{(\ell)} > Q_p. \end{cases}$$

Indeed, inside the representable interval $\bar{w}_i^{(\ell)} = w_i^{(\ell)}$ and hence does not vary with a_ℓ ; outside it, the clipped weight equals respectively $a_\ell Q_n$ or $a_\ell Q_p$. Since $\beta_i^{(\ell)*}$ is the selected subgradient of Φ_ℓ with respect to $\bar{w}_i^{(\ell)}$ at fixed a_ℓ , the subgradient chain rule contributes

$$\sum_{i \in \mathcal{I}_\ell} \beta_i^{(\ell)*} \frac{\partial \bar{w}_i^{(\ell)}}{\partial a_\ell}.$$

Substituting the preceding piecewise derivative produces the final two sums in Eq. (48).

Equations (44) and (46) provide the latent-weight and step-size components of the task-loss gradient introduced in the task-gradient definition of Section 2.2. Collecting these components gives

$$\nabla_{(w,a,\theta_{\text{fp}})} \mathcal{L}_{\text{task}} = \left((\nabla_{w_i^{(\ell)}} \mathcal{L}_{\text{task}})_{\ell \in \Lambda, i \in \mathcal{I}_\ell}, (\nabla_{a_\ell} \mathcal{L}_{\text{task}})_{\ell \in \Lambda}, \nabla_{\theta_{\text{fp}}} \mathcal{L}_{\text{task}} \right). \quad (49)$$

Likewise, Eqs. (47) and (48) provide the corresponding components of the selected PRESTO subgradient introduced in Section 2.2, so

$$\nabla_{\text{PRESTO}} = \left(((\nabla_{\text{PRESTO}})_{w_i^{(\ell)}})_{\ell \in \Lambda, i \in \mathcal{I}_\ell}, ((\nabla_{\text{PRESTO}})_{a_\ell})_{\ell \in \Lambda}, 0 \right). \quad (50)$$

Finally, combining Eqs. (49) and (50), the direction passed to the optimizers during training is

$$\nabla_{\text{train}} \mathcal{J} := \nabla_{(w,a,\theta_{\text{fp}})} \mathcal{L}_{\text{task}} + \nabla_{\text{PRESTO}}. \quad (51)$$

This direction lies in $\partial_{(w,a,\theta_{\text{fp}})} \mathcal{J}$ when the dual is at an optimum; with the truncated dual actually used it is the inexact proxy characterized in Appendix C.12.

C.14 ORDER OF THE OUTER UPDATE

Let k index the steps of the outer optimizer. At the beginning of step k , w_k denotes the collection of latent quantized weights, a_k the collection of learned step sizes, and $\theta_{\text{fp},k}$ the parameters retained in full precision. All quantities entering the gradients at step k must be evaluated from this same parameter state, in the following order:

- (1) construct the levels $v_{b,k}^{(\ell)} = a_{\ell,k} q_b$;
- (2) perform the forward pass using $\hat{w}(w_k, a_k)$ and compute the task-loss gradients;
- (3) solve the inner problem using $\bar{w}_k$, a_k , and the levels v_k , obtaining x_k^* , y_k^* , μ_k , and β_k^* ;
- (4) form the training direction of Appendix C.13 from the task-loss components in Eqs. (44) and (46) and the selected PRESTO components in Eqs. (47) and (48);
- (5) update w_k , a_k , and $\theta_{\text{fp},k}$ with their respective optimizers, producing the state indexed by $k + 1$.

In this list, v_k collects all real-valued levels, $\bar{w}_k$ collects all clipped latent weights, and x_k^* , y_k^* , μ_k , and β_k^* collect the corresponding solutions or derivatives across all tensors and weights. In particular, the dual solver at step k must use the levels generated by a_k , because those same levels produced the quantized weights in the forward pass. Updating a_k before solving the inner problem would instead combine the clipped weights and task gradients associated with one codebook with PRESTO gradients associated with another. The resulting quantity would therefore not be a gradient of the single parameter state appearing in Eq. (8).

C.15 ALGORITHMS AND ADDITIONAL DIAGNOSTICS

Algorithm 1 states the exact per-weight solver of Appendix C.3 and Algorithm 2 the training loop that calls it. The two are written for a single tensor and a single optimizer step respectively; in the implementation the per-weight loop of Algorithm 1 is a vectorized sweep over the weights of the tensor, and Algorithm 2 runs under DDP with the dual state replicated.

Algorithm 1 Exact inner solve for one tensor at a fixed multiplier. Cost $O(C + n_\ell C)$; no iteration.

Require: clipped weights $\bar{w} \in \mathbb{R}^{n_\ell}$, levels $v_b = a_\ell q_b$ for $b \in \mathcal{B}$, multiplier $\xi \in \mathbb{R}^B$, coefficients T_1, T_2

Ensure: optimal masses y^* , relaxed counts $\hat{c}$, weight subgradients β^*

- 1: build the lower convex hull of $\{(v_b, \xi_b)\}_{b \in \mathcal{B}}$ by a monotone chain $\triangleright O(C)$, once per tensor per dual iteration
 - 2: **for** each weight i **do**
 - 3: $V_i^{\max} \leftarrow \max\{v_b : \bar{w}_i v_b \geq 0\}$; $y_i^{\min} \leftarrow |\bar{w}_i| / V_i^{\max}$ (and 0 if $\bar{w}_i = 0$)
 - 4: $\mathcal{Y} \leftarrow \{y_i^{\min}, 1\} \cup \{|\bar{w}_i| / v_{(k)}\} \cap [y_i^{\min}, 1]$ $\triangleright$ facet breakpoints, Eq. (24)
 - 5: locate the facet active at $y = 1$ by binary search $\triangleright O(\log C)$
 - 6: **for** each subinterval $[y_-, y_+]$ induced by $\mathcal{Y}$, in order **do**
 - 7: read (μ_i, s_i) of the facet active there $\triangleright$ Eq. (22)
 - 8: **if** $s_i + T_2 > 0$ **then** $y^{\text{cand}} \leftarrow |\bar{w}_i| \sqrt{T_1 / (s_i + T_2)}$ $\triangleright$ Eq. (26)
 - 9: **end if**
 - 10: evaluate G_i at y^{cand} if it lies in $[y_-, y_+]$, and at y_- and y_+ $\triangleright$ Eq. (25)
 - 11: **end for**
 - 12: $y_i^* \leftarrow \arg \min$ over all evaluated points; $z_i^* \leftarrow 1 - y_i^*$
 - 13: place the mass on the at most two hull-adjacent symbols that represent $\bar{w}_i$ at y_i^* ; accumulate into $\hat{c}$
 - 14: **if** $\bar{w}_i = 0$ **then** $y_i^* \leftarrow 0$; $\beta_i^* \leftarrow 0$ $\triangleright$ the ratio is 0/0; see Appendix C.3
 - 15: **else** $\beta_i^* \leftarrow \mu_i + 2T_1 \bar{w}_i / y_i^*$, or the boundary selection of Appendix C.6 if $|\bar{w}_i| = V_i^{\max}$ $\triangleright$ Eq. (12)
 - 16: **end if**
 - 17: **end for**
-

Algorithm 2 One PRESTO optimizer step. The regularizer enters the same backward pass as the task loss, so weights, step sizes and symbol distribution move together.

Require: latent weights w , step sizes a , warm-started multipliers ξ , step counter k

- 1: $\hat{w} \leftarrow \text{LSQ}(w, a)$; accumulate $\nabla \mathcal{L}_{\text{task}}$ by backpropagation with the straight-through estimator
- 2: **if** $T_3 > 0$ **and** $k \bmod \text{entropy_every} = 0$ **and** the entropy warm-up is over **then**
- 3: **for** $t = 0$ **to** $N_{\text{dual}} - 1$ **do** ▷ FISTA-style ascent, Appendix C.8
- 4: $(y^*, \hat{c}, \beta^*) \leftarrow \text{Algorithm 1 at } \xi^{(t)}$ ▷ solved at the current multiplier
- 5: $c_b^* \leftarrow \text{clip}(2\xi_b^{(t)}/T_3 - 1/\ln 2, c_{\min}, n_\ell)$ ▷ count subproblem, closed form
- 6: $g \leftarrow (\hat{c} - c^*)/n_\ell$
- 7: $\tilde{\xi} \leftarrow \xi^{(t)} + \frac{\tau_t - 1}{\tau_{t+1}}(\xi^{(t)} - \xi^{(t-1)})$ ▷ extrapolation *after* the solve
- 8: $\xi^{(t+1)} \leftarrow \text{clip}(\tilde{\xi} + \eta_\xi g, \xi_{\min}, \xi_{\max})$ ▷ η_ξ a fraction of $\xi_{\max} - \xi_{\min}$
- 9: **end for**
- 10: add β^* to the weight gradient and the matching envelope term to the step-size gradient ▷ Appendix C.13
- 11: **else if** $T_3 = 0$ **then**
- 12: add the closed-form gradient, which reduces to $2T_1 \bar{w}$ on the in-range weights when $T_2 = 0$
- 13: **else**
- 14: add nothing: with $T_3 > 0$ the regularizer reaches the weights only on the steps that call the dual ▷ see Section 4.3
- 15: **end if**
- 16: step the weight optimizer and the separate step-size optimizer; carry ξ to the next call

Byte breakdown. Every ratio in this paper decomposes into three streams, reported per run by the serializer and summarized here for the ResNet-18 block: at 61.83% sparsity the support mask costs 2.755% of the FP32 tensor bytes, the value stream 2.066%, and the metadata 0.000197%, the last being one fp32 step size per tensor over twenty-one tensors. The metadata term is negligible under per-tensor grids and becomes a hard floor under per-channel ones, 0.599% on EfficientNet-B0 with its eighty-two tensors, which is the quantity Appendix B.7 prices.

D CALIBRATING THE COEFFICIENTS

None of the three coefficients can be derived from the network in closed form, and we tried to derive them before settling on measurement. This appendix records the procedure that does work, because it is the part of the method that a practitioner has to repeat.

The dose is measured, then transported by its integral. Three analytic models for the sparsity coefficient failed in sequence on AlexNet. The first derived the force from the dual threshold and turned out to be inert, by three to four orders of magnitude. The second was calibrated on the inner asymptote $2\sqrt{T_1 T_2}$ while the binding regime was in fact the floor branch, on which the force is linear in T_2 , so it overshot by the square of the intended factor. The third corrected the force law and was still ten times too strong, because the dominant channel was not present in the equation at all: the sparsity term inflates the step sizes and widens the interval that is mapped to the zero symbol instead of reshaping the latent distribution, and widening that cell is the cheaper way of obtaining the missing mass. What replaced the derivation is a deliberately overdosed ramp that is read one epoch at a time, so that the ramp fraction itself becomes the dose axis.

Exposure, not the coefficient, is what transfers. A ramp row reports the exposure accumulated up to that epoch, not a coefficient held for a whole run, and reproducing that row at fixed dose requires redoing the integral. Writing $S = \sum_e \eta(e)/\eta_0$ for the schedule sum of the learning rate over the run, the quantity that transfers between two runs is $T \cdot S$. On EfficientNet-B0 the ramp row that reads $T_2 = 10^{-6}$ at epoch six reproduces on a twenty-epoch cosine only at 3.16×10^{-7} , a factor of 3.2, and the row one epoch earlier at 2.28×10^{-7} , a factor of 4.4; the knee taken from the later row, carried to a thirty-epoch cosine whose schedule sums are 10.9 and 16.2, needs 2.1×10^{-7} , which is the dose of Table 2. Every dose reported in this paper was transported in this way.

The entropy coefficient extrapolates on a log scale, until it does not. Regressing the two sixty-epoch ResNet-18 rows of Tables 1 and 6 on $\ln T_3$ gives -1.29 points of size and -0.81 points of accuracy per unit. The size prediction holds well outside the fitted range: at $T_3 = 2.6 \times 10^{-7}$, two and a half times the largest fitted dose, it predicts 3.00% against the 3.02% measured, and it does so across budgets, since the deep point was run for a hundred epochs. The accuracy prediction does not: it predicts -0.90 where we measure -1.404 , because the marginal cost more than doubles once the near-lossless regime is left. We report both predictions because the asymmetry between them is the useful part, in the sense that the size can be planned in advance while the accuracy has to be verified by running the configuration.

What each coefficient is for. T_1 is the perspective ridge and is held at 10^{-5} everywhere, and although the term ablation shows that it does nothing on its own, it exists in order to keep the inner problem well posed as $y_i \rightarrow 0$. T_2 is dosed to a target sparsity that is read off a ramp, while T_3 is dosed to a target size, and in that case the dual step has to be relative, because otherwise the coefficient does not control the outcome at all.

E WHAT IS HELD IDENTICAL ACROSS ARCHITECTURES

Every claim in this paper is a difference between runs, so the parts of the recipe that are not the difference under study have to be identical, and Table 12 records which parts are held fixed and which are not.

Table 12: Left: held identical on every run reported. Right: varied per architecture, with the reason. Nothing else differs.

Identical everywhere	Varied, and why
quantizer LSQ, uniform grid	learning rate and its floor: tuned per network
step-size init by MSE grid search	batch size: memory, and the teacher when distilling
no gradient scaling on the step sizes	epoch budget: compute, stated in every row
no external pruning rule of any kind	distillation: on both transformers at
dual step 1.3×10^{-2} , relative	$\alpha = 0.9$ and on AlexNet and
$N_{\text{dual}} = 3$, six on two ViT rows	EfficientNet-B0 at $\alpha = 0.5$, off
dual called every 4 optimizer steps	on both ResNets
one warm-up epoch before the entropy term	layer_C: eight bits on the input
step-size rate follows the weight cosine	convolutions of AlexNet, and on the
per-tensor step sizes in every run of	depthwise tensors of EfficientNet-B0
Tables 1–6	step-size rate in relative mode on AlexNet
$T_1 = 10^{-5}$ in every PRESTO run	and EfficientNet-B0, absolute elsewhere
and 0 in the $(0, 0, 0)$ controls	per-channel step sizes on the superseded
	controls of Table 4 only

Three of the entries deserve a comment. The step-size rate is expressed as an absolute value on the ResNets and the transformers and as a fraction of the tensor’s own initial step on AlexNet and EfficientNet-B0; the two coincide to within 25% on AlexNet, whose mean initial step is 0.0074 against the 0.01 the absolute value was calibrated on, but differ by a factor of eight on EfficientNet-B0, whose steps are four times larger, and there the relative form is the correct one because it makes the per-step displacement uniform across tensors. The eight-bit tensors are always a stated minority of the parameters and always counted at their true cost in the packet: 0.07% of AlexNet and 3.7% of EfficientNet-B0. And $N_{\text{dual}} = 3$ holds for every run except the two ViT-B/16 frontier points, which use 6; Appendix A.6 reports the matched comparison at both values.

The dual solver dominates the cost and its complexity is $O(N_{\text{dual}}(B + n_{\ell}C))$ per tensor and per call, of which the hull construction is the $O(B)$ part and the per-weight sweep the rest. Measured per call at $N_{\text{dual}} = 3$ this is 48 milliseconds per tensor plus 9 nanoseconds per weight, fitted on three networks spanning 21 to 54 tensors and 11.7M to 86.3M weights, the per-tensor term carrying most of the cost everywhere in that range; Appendix A.6 gives the wall-clock table and the reason the two ViT-B/16 rows cannot be mixed into such a fit.

F GLOSSARY OF NOTATION AND ACRONYMS

Table 13 collects the mathematical objects and the principal acronyms used throughout the paper. Superscript (ℓ) identifies a quantized tensor, subscript i identifies a weight within that tensor, subscript b identifies a nonzero codebook symbol, superscript $*$ denotes a selected optimal value, and superscript t or subscript k denotes an algorithmic iteration as specified below.

Table 13: Glossary of the notation and acronyms used in the paper.

Symbol or acronym	Meaning
<i>Data, network, and indexing</i>	
$\mathcal{D}$	Training dataset $\{(x_j, t_j)\}_{j=1}^N$.
N	Number of examples in the training dataset.
j	Index of a training example.
x_j	Input of training example j ; distinct from the assignment variables $x_{i,b}^{(\ell)}$.
t_j	Target associated with input x_j .
$\mathcal{X}, \mathcal{Y}$	Input space and target space, respectively.
K	Number of classification classes; distinct from the per-weight value function K_i .
$f(\cdot; \theta)$	Neural network parameterized by θ and returning K logits.
θ	Collection of all network parameters, partitioned as (w, θ_{fp}) .
w	Collection of latent weight tensors selected for quantization.
θ_{fp}	Biases, normalization parameters, and other parameters retained in floating-point precision.
M	Number of quantized weight tensors.
Λ	Tensor-index set $\{1, \dots, M\}$.
ℓ	Index of a quantized tensor, with $\ell \in \Lambda$.
$w^{(\ell)}$	Latent weight vector of tensor ℓ .
n_ℓ	Number of scalar weights in tensor ℓ .
$\mathcal{I}_\ell$	Weight-index set $\{1, \dots, n_\ell\}$.
i	Index of a scalar weight within tensor ℓ .
ℓ_{task}	Per-example task-loss function; distinct from the tensor index ℓ .
$\mathcal{L}_{\text{task}}$	Empirical task loss over $\mathcal{D}$.
$\mathcal{R}_{\text{PRESTO}}$	Complete PRESTO regularizer $\sum_{\ell \in \Lambda} \Phi_\ell$.
$\mathcal{J}$	Complete outer training objective.
<i>Learned-step quantization (LSQ)</i>	
Q_n, Q_p	Minimum and maximum integer quantization codes.
$\mathcal{Q}$	Integer codebook $\{q \in \mathbb{Z} : Q_n \leq q \leq Q_p\}$.
q	Generic integer code in $\mathcal{Q}$.
C	Total number of symbols, $C = \mathcal{Q} $.
a_ℓ	Learned quantization step size of tensor ℓ .
a	Collection $\{a_\ell\}_{\ell \in \Lambda}$ of all step sizes.
$r_i^{(\ell)}$	Normalized latent weight $w_i^{(\ell)} / a_\ell$.
$\bar{w}_i^{(\ell)}(a_\ell)$	Latent weight clipped to the representable interval of tensor ℓ .
$\bar{w}^{(\ell)}$	Vector of clipped latent weights in tensor ℓ .
$\hat{w}_i^{(\ell)}$	Quantized weight used in the forward pass.
$\hat{w}$	Collection of all quantized weight tensors.
<i>PRESTO inner problem</i>	
$\mathcal{B}$	Set $\mathcal{Q} \setminus \{0\}$ of nonzero symbols.
B	Number of nonzero symbols, $B = \mathcal{B} = C - 1$.
b	Index of a nonzero symbol in $\mathcal{B}$.
q_b	Integer code associated with nonzero symbol b .
$v_b^{(\ell)}(a_\ell)$	Real-valued level $a_\ell q_b$ of symbol b in tensor ℓ .
$v^{(\ell)}, v$	Vector of real-valued nonzero levels, with the tensor superscript suppressed locally when unambiguous.
$x_{i,b}^{(\ell)}$	Nonnegative mass assigning weight i of tensor ℓ to nonzero symbol b .

Continued on the next page

Table 13 continued

Symbol or acronym	Meaning
$x_i^{(\ell)}, x^{(\ell)}$	Assignment masses for one weight and for the complete tensor, respectively.
$y_i^{(\ell)}$	Total mass representing weight i through nonzero symbols.
$z_i^{(\ell)}$	Mass $1 - y_i^{(\ell)}$ assigned to the zero symbol; distinct from the temporary dual-update vector $z_b^{(\ell, t+1)}$.
$c_b^{(\ell)}$	Relaxed count of nonzero symbol b in tensor ℓ .
T_1	Coefficient of the perspective ridge term.
T_2	Coefficient penalizing mass assigned to nonzero symbols.
T_3	Coefficient of the entropy-related count term.
$\Phi_\ell(\bar{w}^{(\ell)}, a_\ell)$	Optimal value of the tensor-level PRESTO inner problem.
<i>Lagrangian relaxation and decomposition</i>	
$\xi_b^{(\ell)}$	Unrestricted Lagrange multiplier of the count-defining constraint for symbol b .
$\xi^{(\ell)}$	Vector in $\mathbb{R}^B$ collecting the nonzero-symbol count multipliers of tensor ℓ .
$\mathcal{L}_\ell$	Partial Lagrangian obtained by dualizing the count constraints $c_b^{(\ell)} = \sum_{i \in \mathcal{I}_\ell} x_{i,b}^{(\ell)}$ for every $b \in \mathcal{B}$.
$\varphi_{\bar{w}^{(\ell)}, a_\ell}^{(\ell)}$	Concave Lagrangian dual function obtained by minimizing $\mathcal{L}_\ell$ over the retained primal constraints.
$\mathcal{F}_i^{(\ell)}$	Feasible set of the per-weight assignment and mass variables.
$\mathcal{L}_\ell^{\text{full}}$	Inner Lagrangian including both the count and representation multipliers.
<i>Per-weight solution</i>	
$K_i(y_i; \bar{w}_i, \xi, v)$	Optimal value of the linear assignment subproblem for weight i at fixed nonzero mass y_i .
$u_{i,b}$	Normalized assignment mass $x_{i,b}/y_i$; distinct from the generic scalar u in the perspective definition.
$\tilde{w}_i$	Rescaled target $\bar{w}_i/y_i$.
P_b	Point (v_b, ξ_b) used to construct the lower convex hull.
E_l, E_r	Leftmost and rightmost vertices of the lower convex hull.
b_-, b_+	Indices of the endpoints of the active lower-hull facet.
μ_i	Slope of the active lower-hull facet and representation-constraint multiplier under the paper’s sign convention.
s_i	Intercept of the active lower-hull facet; distinct from the dual residual $s_b^{(\ell)}$.
$V_i^{\max}$	Largest representable magnitude having the same sign as $\bar{w}_i$.
$y_i^{\min}$	Smallest nonzero mass capable of representing $\bar{w}_i$.
$G_i(y_i)$	Scalar per-weight objective after eliminating the assignment variables.
y_i^{cand}	Interior stationary candidate for minimizing G_i .
y_i^*	Selected minimizer of the scalar per-weight problem.
z_i^*	Optimal zero-symbol mass $1 - y_i^*$.
G_i^*	Optimized per-weight value obtained after minimizing G_i over y_i .
$e, v_e^{(\ell)}$	Index and real-valued level of an active extreme codebook symbol.
<i>Sensitivity and training gradients</i>	
$\mu_i^{(\ell)}$	Multiplier of the representation constraint for weight i in tensor ℓ .
$\beta_i^{(\ell)*}$	Selected PRESTO subgradient with respect to the clipped latent weight $\bar{w}_i^{(\ell)}$ at fixed a_ℓ .
$\delta_i^{(\ell)}$	Upstream task-loss gradient $\partial \mathcal{L}_{\text{task}} / \partial \hat{w}_i^{(\ell)}$.
$h(r)$	Piecewise LSQ surrogate derivative of a quantized weight with respect to its step size.
$\nabla_{w_i^{(\ell)}} \mathcal{L}_{\text{task}}, \nabla_{a_\ell} \mathcal{L}_{\text{task}}$	Task-loss components with respect to a latent weight and a step size.
∇_{PRESTO}	Direction used for training: the element of $\partial \mathcal{R}_{\text{PRESTO}}$ selected by the procedure at a dual optimum, and an inexact proxy for it otherwise.
$(\nabla_{\text{PRESTO}})_{w_i^{(\ell)}}$	Latent-weight component of the selected PRESTO subgradient.
$(\nabla_{\text{PRESTO}})_{a_\ell}$	Step-size component of the selected PRESTO subgradient.

Continued on the next page

Table 13 continued

Symbol or acronym	Meaning
$\nabla_{\text{train}} \mathcal{J}$	Selected (sub)gradient of the complete objective passed to the network optimizers.
<i>Dual solver and optimization order</i>	
$\hat{c}_b^{(\ell)}$	Count reconstructed by summing the optimal per-weight assignment masses.
$c_{b,\text{unc}}^{(\ell)}$	Unconstrained stationary count returned by the exponential relation.
$c_b^{(\ell)*}$	Count used by the dual solver after clipping $c_{b,\text{unc}}^{(\ell)}$.
$c_{\min}$	Numerical lower bound imposed on the count solution.
$s_b^{(\ell)}$	Unnormalized residual $\hat{c}_b^{(\ell)} - c_b^{(\ell)*}$ of the relaxed count constraint; distinct from the facet intercept s_i .
N_{dual}	Number of dual iterations performed per solver call.
t	Iteration index of the dual solver, $t \in \{0, \dots, N_{\text{dual}} - 1\}$.
η_{ξ}	Dual supergradient-ascent step size.
$g_b^{(\ell,t)}$	Tensor-size-normalized dual supergradient at iteration t .
τ_t	FISTA-style momentum parameter.
$\xi^{(\ell,t)}$	Momentum-extrapolated multiplier vector.
$z_b^{(\ell,t+1)}$	Unclipped multiplier update before componentwise clipping; distinct from the zero-symbol mass $z_i^{(\ell)}$.
$\xi_{\min}^{(\ell)}, \xi_{\max}^{(\ell)}$	Lower and upper numerical bounds imposed on the dual multipliers.
Ξ_{ℓ}	Box $[\xi_{\min}^{(\ell)}, \xi_{\max}^{(\ell)}]^B$ onto which the multiplier is projected; the ascent runs on the dual restricted to it.
$\Phi_{\ell}^{\square}, \varphi_{\ell}^{\square}$	Inner problem and partial dual with the counts confined to $[c_{\min}, n_{\ell}]$, which is what the solver poses.
k	Iteration index of the outer network optimizer.
$w_k, a_k, \theta_{\text{fp},k}$	Weight, step-size, and floating-point parameter collections at the start of outer step k .
$v_k, \bar{w}_k$	Collections of real-valued levels and clipped latent weights at outer step k .
$x_k^*, y_k^*, \mu_k, \beta_k^*$	Per-weight solutions and sensitivity quantities used at outer step k .
<i>Appendix-only KKT notation</i>	
$\rho_i^{(\ell)}$	Multiplier of the per-weight mass constraint in the boundary KKT derivation.
$\gamma_{i,b}^{(\ell)}$	Nonnegative multiplier of the constraint $-x_{i,b}^{(\ell)} \leq 0$.
$\mathcal{L}_{i,\ell}^{\text{KKT}}$	Per-weight KKT Lagrangian used in the boundary derivation.
<i>Principal acronyms and reporting terms</i>	
PRESTO	Perspective-Based Entropy–Sparsity Training Optimization.
QAT	Quantization-aware training.
LSQ	Learned Step Size Quantization.
STE	Straight-through estimator.
FISTA	Fast Iterative Shrinkage–Thresholding Algorithm; the implementation uses a FISTA-style dual update.
KKT	Karush–Kuhn–Tucker optimality conditions.
FP32	32-bit floating-point representation.
W4A32	4-bit weights and 32-bit floating-point activations.
BN	Batch normalization.
Zstd	Zstandard lossless compression.
Δ	Difference between quantized and FP32 top-1 accuracy, measured in percentage points.